



INFORME FINAL DE PROYECTO

“Título Técnico: Implementación de Pong”

(Proyecto de 1 Mes en Lógica Digital o Ensamblador)

Integrantes:

Jorge Eliécer Martelo Martelo - Código T00078391
Anderson Ibáñez Espinoza - Código T00078386
Jesús Manuel Mercado Machado - Código T00078505
Juan Carlos Narváez Castaño - Código T00078519
Omar Yesid Hernández Sotelo - Código T00078505

Docente:

Ing. Daniel Andres Arias Lopez

Universidad Tecnológica de Bolívar

Cartagena de Indias, 19 de mayo de 2026

Índice

1. Introducción y Justificación	3
2. Objetivos	5
2.1. Objetivo General	5
2.2. Objetivos Específicos	5
3. Marco Teórico	6
3.1. Arquitectura del Procesador Intel 8086	6
3.2. Modelo de Segmentación de Memoria	6
3.3. Manejo de la Pila y Llamadas a Subrutinas	6
3.4. Interrupciones del Sistema	6
3.5. Modo de Video 13h y Framebuffer	7
3.6. Control de Audio mediante Temporizador 8253	7
3.7. Detección de Colisiones por Bounding-Box	7
4. Metodología y Procedimiento	8
4.1. Herramientas y Recursos Utilizados	8
4.2. Procedimiento de Desarrollo	8
4.2.1. Fase 1 – Conceptualización y Diseño del Sistema	8
4.2.2. Fase 2 – Configuración del Entorno Gráfico	9
4.2.3. Fase 3 – Implementación del Ciclo Principal y Sincronización	9
4.2.4. Fase 4 – Lógica de Juego: Pelota, Colisiones y Puntuación	9
4.2.5. Fase 5 – Entrada de Teclado y Control de Paletas	10
4.2.6. Fase 6 – Audio, Interfaz de Usuario y Depuración	10
4.3. Diagramas de Flujo	10
4.3.1. Flujo del Ciclo Principal (MAIN)	11
4.3.2. Flujo de MOVE_BALL	12
4.3.3. Flujo de MOVE_PADDLES	12
5. Resultados y Evidencias Visuales	13
6. Discusiones y Mejoras Futuras	15
6.1. Dificultades Encontradas y Soluciones Implementadas	15
6.1.1. Sincronización del Ciclo de Juego	15
6.1.2. Detección de Colisiones con Signo en Registros de 16 Bits	15
6.1.3. Control de Audio sin Bloqueo	16
6.1.4. Gestión de la Paleta Derecha con Teclas de Función	16
6.1.5. Limitación de un Solo Evento de Teclado por Ciclo	16
6.2. Análisis de Estabilidad del Producto	16
6.3. Mejoras Futuras	17
6.3.1. Audio Asíncrono mediante Interrupción de Temporizador	17
6.3.2. Doble Buffer de Video para Eliminar el Parpadeo	17

6.3.3. Inteligencia Artificial para Modo de un Jugador	18
6.3.4. Escalabilidad a Arquitecturas de 32 y 64 Bits	18
6.3.5. Persistencia de Puntuaciones y Tabla de Líderes	18
7. Conclusiones y Referencias	19
7.1. Conclusiones	19
7.2. Referencias	20
A. Anexos	21
A.1. Código Fuente Completo (Documentado)	21
B. Guía de Sustentación del Proyecto	38
B.1. El Gancho Inicial (Obligatorio)	38
B.2. Reglas de la Exposición	38
B.3. Rúbrica de Calificación (Informe Final 20 %)	38
B.4. Rúbrica de Calificación (Sustentación 35 %)	38

1. Introducción y Justificación

El desarrollo de software a nivel de lenguaje ensamblador representa uno de los ejercicios más exigentes y formativos dentro de la disciplina de arquitectura de computadoras. A diferencia de los lenguajes de alto nivel, el ensamblador expone directamente los recursos físicos del procesador —registros, buses, unidades aritmético-lógicas (ALU) y mecanismos de interrupción— obligando al programador a razonar en términos de la microarquitectura subyacente [1].

El presente proyecto consiste en la implementación completa del videojuego clásico *Pong* sobre la arquitectura Intel 8086, utilizando el modo de video gráfico 13h (320×200 píxeles, 256 colores) provisto por la BIOS. El programa gestiona: dos paletas controladas por teclado mediante interrupciones hardware (INT 16h), una pelota con lógica de colisiones por caja delimitadora (*bounding-box*), un sistema de puntuación con detección de ganador, retroalimentación sonora a través del altavoz del sistema (puertos 42h–43h–61h) y un ciclo de juego sincronizado mediante la función de tiempo del DOS (INT 21h, servicio 2Ch).

Relevancia del problema

A pesar de que los procesadores modernos operan con pipelines de múltiples etapas, predicción de saltos y cachés de varios niveles, los principios fundamentales que rigen su funcionamiento —el ciclo *fetch-decode-execute*, el rol del banco de registros y la segmentación de la memoria— son los mismos que gobiernan el 8086 [2]. Programar un videojuego en ensamblador constituye, por tanto, un banco de pruebas privilegiado para comprender cómo el hardware transforma instrucciones de bajo nivel en comportamiento observable: movimiento de objetos en pantalla, respuesta a la entrada del usuario en tiempo real y generación de señales de audio.

Desde la perspectiva de la industria, los sistemas embebidos y los controladores de tiempo real siguen requiriendo programación cercana al hardware. Comprender el flujo de control a nivel de instrucción es, en consecuencia, una competencia vigente y valorada en áreas como el diseño de sistemas operativos, la seguridad informática y la computación de alto desempeño [3].

Conexión con conceptos de clase

- **Registros:** El programa utiliza intensivamente los registros de propósito general del 8086 (AX, BX, CX, DX) para almacenar posiciones, velocidades y resultados de comparaciones. Los registros de segmento (CS, DS, SS y ES) delimitan los cuatro segmentos lógicos del modelo de memoria del 8086.
- **ALU:** Las operaciones aritméticas (ADD, SUB, NEG, INC, DIV) y lógicas (AND, OR, XOR, CMP) ejecutadas por la ALU son el mecanismo que actualiza cada fotograma: posiciones de la pelota y las paletas, detección de colisiones y cálculo de puntuación.
- **Bus de datos y de control:** Las instrucciones IN/OUT acceden directamente al bus de I/O para controlar el temporizador 8253 y el altavoz. Las interrupciones (INT

10h, INT 16h, INT 21h) generan ciclos de bus hacia la BIOS y el sistema operativo DOS.

- **Pila (Stack):** El segmento de pila reservado con `DB 64 DUP(' ')` y el uso de `CALL/RET` ilustran el mecanismo de llamada a subrutinas mediante el stack pointer (SP) y el puntero de instrucción (IP).

2. Objetivos

2.1 Objetivo General

Desarrollar un videojuego funcional de Pong en lenguaje ensamblador Intel 8086 que integre gestión de video en modo gráfico, control de entrada por teclado, detección de colisiones, sistema de puntuación y retroalimentación sonora, demostrando el dominio de los recursos de bajo nivel de la arquitectura x86.

2.2 Objetivos Específicos

1. Diseñar la organización de memoria del programa (segmentos de pila, datos y código) conforme al modelo de segmentación del procesador 8086, definiendo las variables de estado del juego —posiciones, velocidades, contadores y marcadores— en el segmento de datos.
2. Implementar el ciclo principal del juego mediante un lazo de control sincronizado con el servicio de tiempo del sistema (INT 21h / función 2Ch), garantizando una frecuencia de actualización de fotogramas determinista al detectar cambios en las centésimas de segundo del reloj del sistema.
3. Programar las subrutinas de entrada/salida para interacción con el usuario, incluyendo la lectura de pulsaciones de teclado mediante INT 16h, el modo de polling sin bloqueo (servicio 01h) y la escritura de texto en pantalla mediante INT 21h (servicio 09h) e INT 10h (servicio 02h para posicionamiento del cursor).
4. Implementar el sistema de renderizado gráfico en modo 13h (320×200 píxeles) utilizando la interrupción INT 10h (servicio 0Ch) para el trazado píxel a píxel de la pelota, las paletas y la línea central punteada.
5. Desarrollar el algoritmo de detección de colisiones por bounding-box para la interacción entre la pelota y las paletas, así como la lógica de rebote en los límites verticales de la pantalla.
6. Integrar retroalimentación sonora mediante el control directo del temporizador 8253 (puertos 42h y 43h) y el altavoz del sistema (puerto 61h), generando tonos diferenciados para los eventos de rebote en paleta, rebote en pared y punto anotado.
7. Validar la correcta transición entre los estados del juego (menú principal, partida activa, pantalla de Game Over y reinicio), implementados como una máquina de estados finita gestionada mediante saltos condicionales e incondicionales (JMP, JE, JGE).

3. Marco Teórico

3.1 Arquitectura del Procesador Intel 8086

El Intel 8086 es un microprocesador de 16 bits introducido en 1978 que establece la base del conjunto de instrucciones x86, aún dominante en la computación de propósito general [4]. Su arquitectura se organiza en torno a un banco de registros de 16 bits —AX, BX, CX, DX (registros de datos), SI, DI (índice), SP, BP (punteros de pila y base) y los registros de segmento CS, DS, ES, SS— y un bus de datos externo de 16 bits con bus de direcciones de 20 bits, que habilita un espacio de direccionamiento físico de 1 MB [2].

Patterson y Hennessy describen la unidad de procesamiento como un ciclo continuo de tres fases: búsqueda de instrucción (*fetch*), decodificación (*decode*) y ejecución (*execute*) [1]. En el 8086 esta secuencia se implementa con una cola de prebúsqueda (*prefetch queue*) de 6 bytes que solapa parcialmente la búsqueda de la siguiente instrucción con la ejecución de la actual, anticipando el concepto de *pipeline* de instrucciones.

3.2 Modelo de Segmentación de Memoria

El 8086 divide la memoria en segmentos de hasta 64 KB cada uno. La dirección física se calcula como:

$$\text{Dirección Física} = (\text{Registro de Segmento} \times 16) + \text{Desplazamiento}$$

El programa implementa explícitamente los tres segmentos canónicos: **STACK** (64 bytes reservados con `DB 64 DUP(' ')`), **DATA** (variables de estado) y **CODE** (procedimientos). La directiva `ASSUME CS:CODE, DS:DATA, SS:STACK` informa al ensamblador de la asociación entre registros de segmento y segmentos lógicos [2].

3.3 Manejo de la Pila y Llamadas a Subrutinas

La pila en el 8086 crece hacia direcciones bajas. La instrucción **CALL** etiqueta decrementa SP en 2 y empuja la dirección de retorno (IP); **RET** restaura IP desde el tope de la pila [1]. El programa estructura toda su lógica en procedimientos (`PROC NEAR/ENDP`) que se invocan con **CALL** y retornan con **RET**, evidenciando el modelo de activación de subrutinas descrito en la literatura de arquitectura de computadoras.

3.4 Interrupciones del Sistema

Las interrupciones son el mecanismo principal de comunicación entre el software de aplicación y el hardware en la arquitectura PC/XT [3]. Al ejecutarse una instrucción `INT n`, el procesador: (1) empuja los flags, CS e IP en la pila, (2) deshabilita las interrupciones (`IF=0`), y (3) transfiere el control al manejador cuya dirección se obtiene de la tabla de vectores de interrupción en `0000:0000h` [4].

El proyecto utiliza las siguientes interrupciones:

- **INT 10h (Servicios de Video BIOS):** Servicio 00h para establecer el modo de video 13h (320×200, 256 colores), servicio 02h para posicionar el cursor de texto y servicio 0Ch para escribir un píxel en modo gráfico.
- **INT 16h (Servicios de Teclado BIOS):** Servicio 00h para leer un carácter con bloqueo y servicio 01h para consultar el estado del buffer de teclado sin bloqueo (*polling*).
- **INT 21h (Servicios del DOS):** Servicio 09h para mostrar cadenas de texto terminadas en '\$', servicio 2Ch para obtener la hora del sistema (horas, minutos, segundos y centésimas) y servicio 4Ch para terminar el programa y retornar el control al sistema operativo.

3.5 Modo de Video 13h y Framebuffer

El modo de video 13h mapea un framebuffer lineal de 64.000 bytes (320 × 200 píxeles, 1 byte por píxel) en la dirección de memoria A000:0000h [6]. Cada byte corresponde a un índice en una paleta de 256 colores. El procedimiento `CLEAR_SCREEN` del programa accede directamente a este buffer cargando ES con 0A000h y usando la instrucción de cadena `REP STOSB` para llenarlo con el valor 00h (negro) en un único ciclo de escritura, lo que constituye la técnica más eficiente disponible en el ISA del 8086 [6].

3.6 Control de Audio mediante Temporizador 8253

El circuito Intel 8253 (Programmable Interval Timer) genera señales de frecuencia configurable a través del canal 2, cuya salida se conecta al altavoz del sistema. La frecuencia se ajusta escribiendo un divisor de 16 bits en el puerto 42h, siendo la frecuencia base del reloj 1.193.180 Hz [5]. La secuencia de activación consiste en: configurar el modo del canal (puerto 43h, valor B6h), cargar el divisor (dos escrituras consecutivas en el puerto 42h, byte bajo primero) y habilitar el altavoz activando los bits 0 y 1 del puerto 61h mediante una operación `OR`.

3.7 Detección de Colisiones por Bounding-Box

La detección de colisiones entre dos rectángulos alineados con los ejes (AABB, *Axis-Aligned Bounding Boxes*) se realiza verificando la ausencia de separación en ambos ejes. Dos rectángulos A y B no colisionan si y solo si se cumple alguna de las siguientes condiciones: $A_{x2} \leq B_{x1}$, $B_{x2} \leq A_{x1}$, $A_{y2} \leq B_{y1}$ o $B_{y2} \leq A_{y1}$ [7]. El procedimiento `MOVE BALL` implementa la versión complementaria: si ninguna de esas condiciones se cumple, se confirma la colisión y se invierte la velocidad horizontal de la pelota (`NEG BALL_VELOCITY_X`).

4. Metodología y Procedimiento

Esta sección describe el proceso completo de diseño, implementación y depuración del videojuego Pong en lenguaje ensamblador 8086, de modo que cualquier persona con conocimientos básicos de arquitectura x86 pueda replicar el proyecto siguiendo los pasos aquí documentados.

4.1 Herramientas y Recursos Utilizados

- **MASM (Microsoft Macro Assembler) / TASM (Turbo Assembler):** Ensamblador utilizado para traducir el archivo fuente `pong.asm` a código objeto. Compatible con la sintaxis de segmentos `SEGMENT/ENDS` y la directiva `ASSUME` empleadas en el proyecto [2].
- **DOSBox 0.74:** Emulador de entorno MS-DOS que permite ejecutar programas COM y EXE compilados para el 8086 en hardware moderno. Reproduce fielmente el modo de video 13h (320×200, 256 colores), las interrupciones de BIOS (INT 10h, INT 16h) y los servicios del DOS (INT 21h) [8].
- **LINK (Microsoft Linker) / TLINK (Turbo Linker):** Enlazador que combina el módulo objeto generado por el ensamblador con las librerías de sistema para producir el ejecutable .EXE final.
- **DEBUG.COM (DOS Debugger):** Utilidad incluida en MS-DOS empleada para inspeccionar el contenido de los registros, ejecutar el programa paso a paso (*single-step*) y verificar el estado de la memoria durante la depuración.
- **Editor de texto plano:** Cualquier editor sin formato (Notepad++, VS Code con extensión ASM, o similar) para escribir y editar el archivo fuente `pong.asm` en codificación ASCII.
- **Documentación de referencia:** Manual técnico del Intel 8086 [4], manual del Intel 8253 [5], referencia de interrupciones de BIOS/DOS (Ralf Brown's Interrupt List) y el libro de programación gráfica de Abrash [6].

4.2 Procedimiento de Desarrollo

El desarrollo se organizó en seis fases secuenciales, desde la conceptualización hasta la validación final del programa.

4.2.1. Fase 1 – Conceptualización y Diseño del Sistema

Se identificaron los componentes mínimos del juego: dos paletas, una pelota, un marcador y un ciclo de juego sincronizado. A partir de esto se definió la estructura de memoria:

1. Establecer el segmento `STACK` con 64 bytes de reserva para las llamadas a subrutinas.

2. Listar en el segmento DATA todas las variables de estado: posiciones (x, y) y velocidades de la pelota, posiciones de las paletas, dimensiones (ancho y alto), contadores de puntos y textos terminados en '\$' para los mensajes en pantalla.
3. Definir las constantes de pantalla en hexadecimal: ancho 140h (320 píxeles) y alto 0C8h (200 píxeles).

4.2.2. Fase 2 – Configuración del Entorno Gráfico

1. Configurar el modo de video gráfico 13h mediante INT 10h (servicio 00h, AL = 13h) al inicio del procedimiento MAIN.
2. Implementar el procedimiento CLEAR_SCREEN: cargar ES con 0A000h (dirección base del framebuffer), inicializar DI = 0, AL = 00h (negro) y CX = 64000, luego ejecutar REP STOSB para limpiar los 320×200 píxeles en una sola operación de cadena.
3. Verificar en DOSBox que la pantalla se muestra en negro limpio antes de continuar con el resto de la lógica.

4.2.3. Fase 3 – Implementación del Ciclo Principal y Sincronización

1. Diseñar el lazo CHECK_TIME que consulta INT 21h / función 2Ch para obtener las centésimas de segundo actuales en DL.
2. Comparar DL con la variable TIME_AUX; si son iguales, reiterar el lazo sin actualizar el estado. Si difieren, actualizar TIME_AUX y ejecutar el ciclo completo: limpiar pantalla → mover pelota → dibujar pelota → dibujar línea central → mover paletas → dibujar paletas → dibujar UI.
3. Esta estrategia de *busy-wait* sobre el reloj del sistema garantiza que el juego avance como máximo 100 veces por segundo, independientemente de la velocidad del procesador emulado en DOSBox.

4.2.4. Fase 4 – Lógica de Juego: Pelota, Colisiones y Puntuación

1. **Movimiento:** En MOVE_BALL, sumar BALL_VELOCITY_X a BALL_X y BALL_VELOCITY_Y a BALL_Y en cada ciclo. Las velocidades son palabras con signo (DW), por lo que valores negativos producen movimiento en sentido contrario sin instrucciones adicionales.
2. **Colisión con bordes verticales:** Comparar BALL_Y con el límite superior (WINDOW_BOUNDS) y con el límite inferior (WINDOW_HEIGHT - BALL_SIZE - WINDOW_BOUNDS). Si se supera cualquiera de los dos, ejecutar NEG BALL_VELOCITY_Y para invertir la trayectoria y llamar a BEEP_WALL.
3. **Colisión con paletas (AABB):** Verificar las cuatro condiciones de solapamiento entre el rectángulo de la pelota y cada paleta. Si se confirma la colisión, ejecutar NEG BALL_VELOCITY_X, llamar a INCREASE_BALL_SPEED y a BEEP_PADDLE.

4. **Punto y reinicio:** Si la pelota supera el borde izquierdo (`BALL_X < WINDOW_BOUNDS`), sumar un punto al Jugador 2 e invocar `RESET_BALL_POSITION` y `RESET_BALL_SPEED`. Simétricamente para el borde derecho. Verificar si el marcador alcanzó 5 puntos para activar el estado de Game Over (`GAME_ACTIVE = 00h`).
5. **Aceleración progresiva:** El procedimiento `INCREASE_BALL_SPEED` incrementa `BOUNCE_COUNT` en cada rebote con paleta y aumenta la velocidad en módulo cada 3 rebotes (verificado mediante división entera y comparación del resto), respetando los límites `BALL_MAX_VEL_X = 0Ch` y `BALL_MAX_VEL_Y = 07h`.

4.2.5. Fase 5 – Entrada de Teclado y Control de Paletas

1. En `MOVE_PADDLES`, consultar el buffer de teclado con `INT 16h / servicio 01h` (sin bloqueo). Si el flag cero (`ZF = 1`) indica que no hay tecla pendiente, saltar directamente a la sección de paleta derecha.
2. Si hay tecla, leerla con `INT 16h / servicio 00h`. Comparar `AL` con los códigos ASCII de 'W'/'w' (`57h/77h`) y 'S'/'s' (`53h/73h`) para la paleta izquierda.
3. Para la paleta derecha, comparar `AH` (código de scan) con `48h` (flecha arriba) y `50h` (flecha abajo).
4. En cada movimiento, verificar los límites de la pantalla y corregir la posición si la paleta sobrepasa `WINDOW_BOUNDS` (arriba) o `WINDOW_HEIGHT - WINDOW_BOUNDS - PADDLE_HEIGHT` (abajo).

4.2.6. Fase 6 – Audio, Interfaz de Usuario y Depuración

1. **Audio:** Implementar los tres procedimientos de sonido (`BEEP_PADDLE` a ≈ 800 Hz, `BEEP_WALL` a ≈ 400 Hz, `BEEP_SCORE` con dos tonos a ≈ 600 Hz y ≈ 1000 Hz) mediante escritura directa a los puertos `43h`, `42h` y `61h`.
2. **UI y menús:** Posicionar el cursor con `INT 10h / servicio 02h` y mostrar cadenas con `INT 21h / servicio 09h` para el menú principal, los marcadores durante la partida y la pantalla de Game Over.
3. **Depuración:** Ejecutar el programa en DOSBox con la variable de entorno de ciclos ajustada (`cycles = 10000`) para observar el comportamiento a velocidad reducida. Usar `DEBUG.COM` para inspeccionar los valores de `BALL_X`, `BALL_Y` y `BALL_VELOCITY_X` con el comando `D DS:0000` y verificar los saltos condicionales con el modo *trace* (T).

4.3 Diagramas de Flujo

A continuación se describen los flujos de control principales del programa. Los diagramas pueden reproducirse con cualquier herramienta de diagramación (draw.io, Lucidchart, Mermaid, etc.) siguiendo la lógica aquí especificada.

4.3.1. Flujo del Ciclo Principal (MAIN)

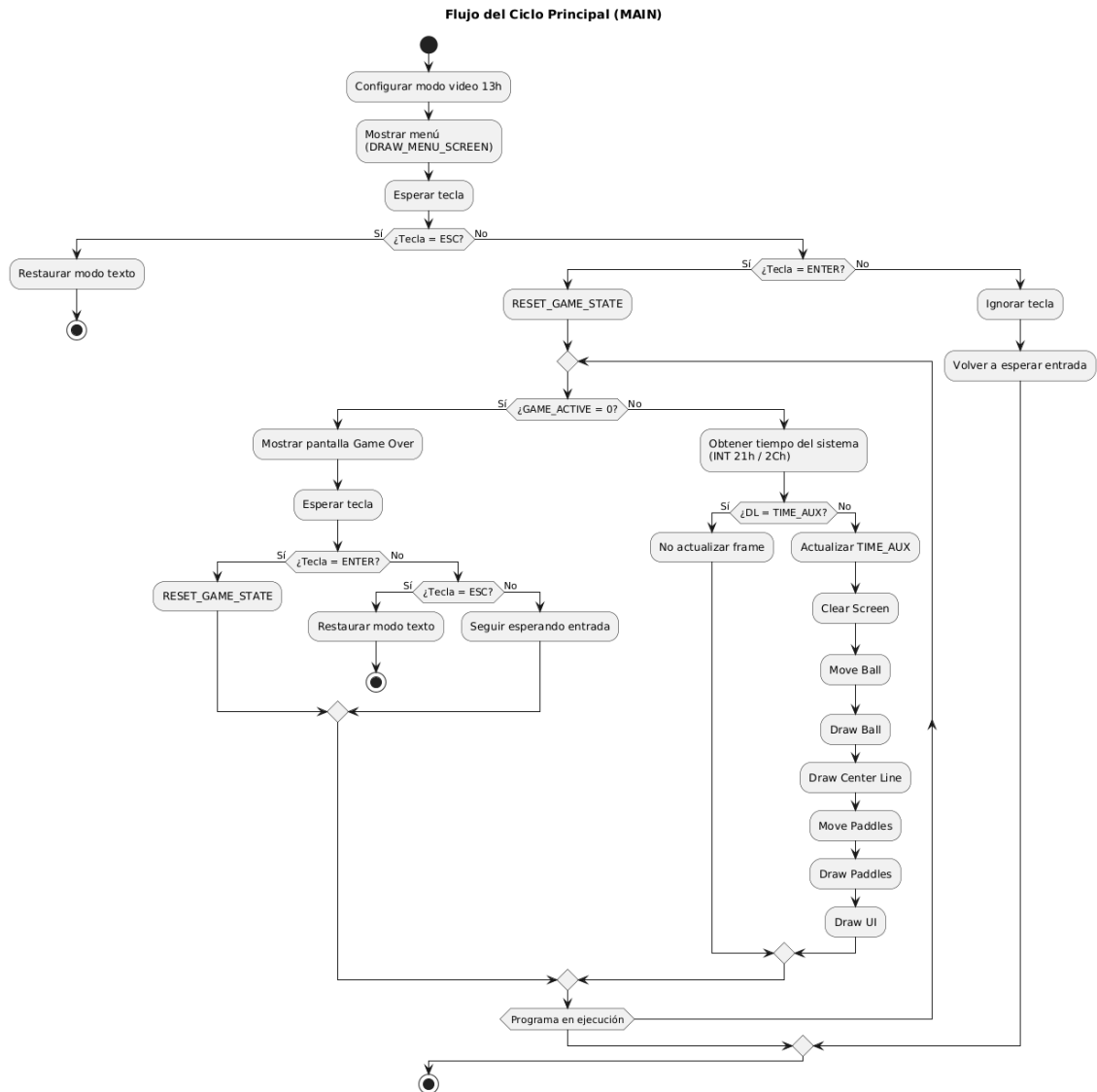


Figura 1: Flujo del ciclo principal

4.3.2. Flujo de MOVE.BALL

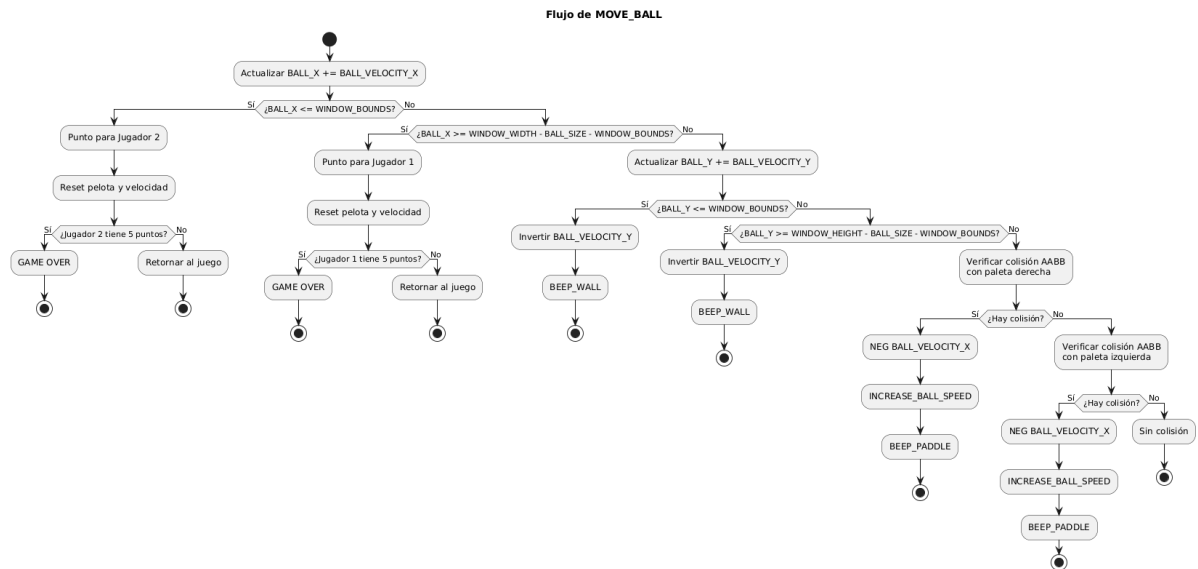


Figura 2: Flujo de MOVE.BALL

4.3.3. Flujo de MOVE.PADDLES

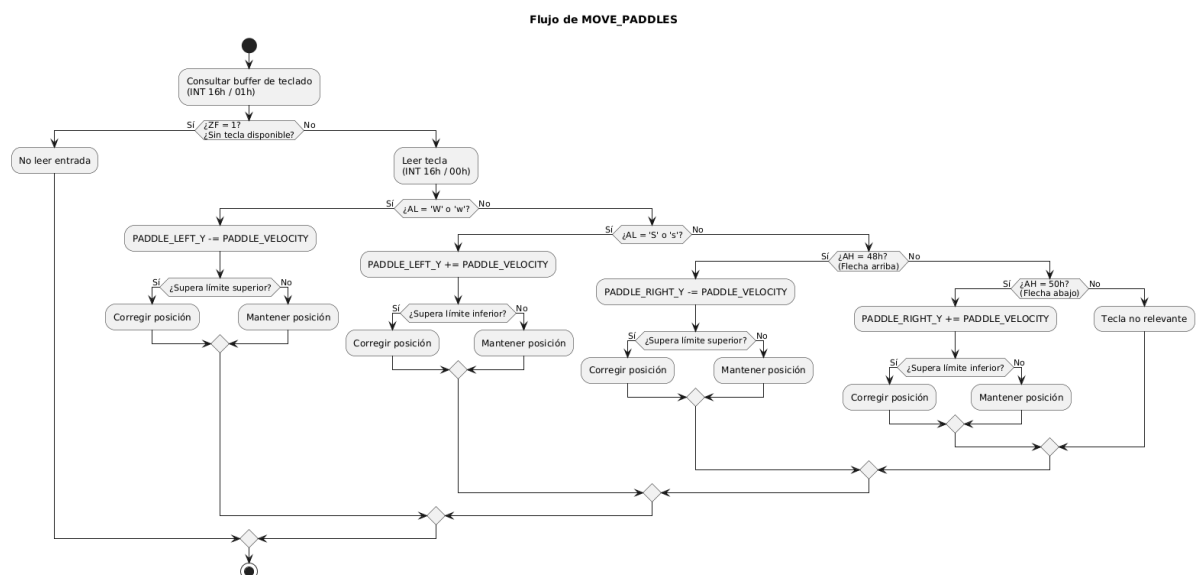


Figura 3: Flujo de MOVE.PADDLES

5. Resultados y Evidencias Visuales

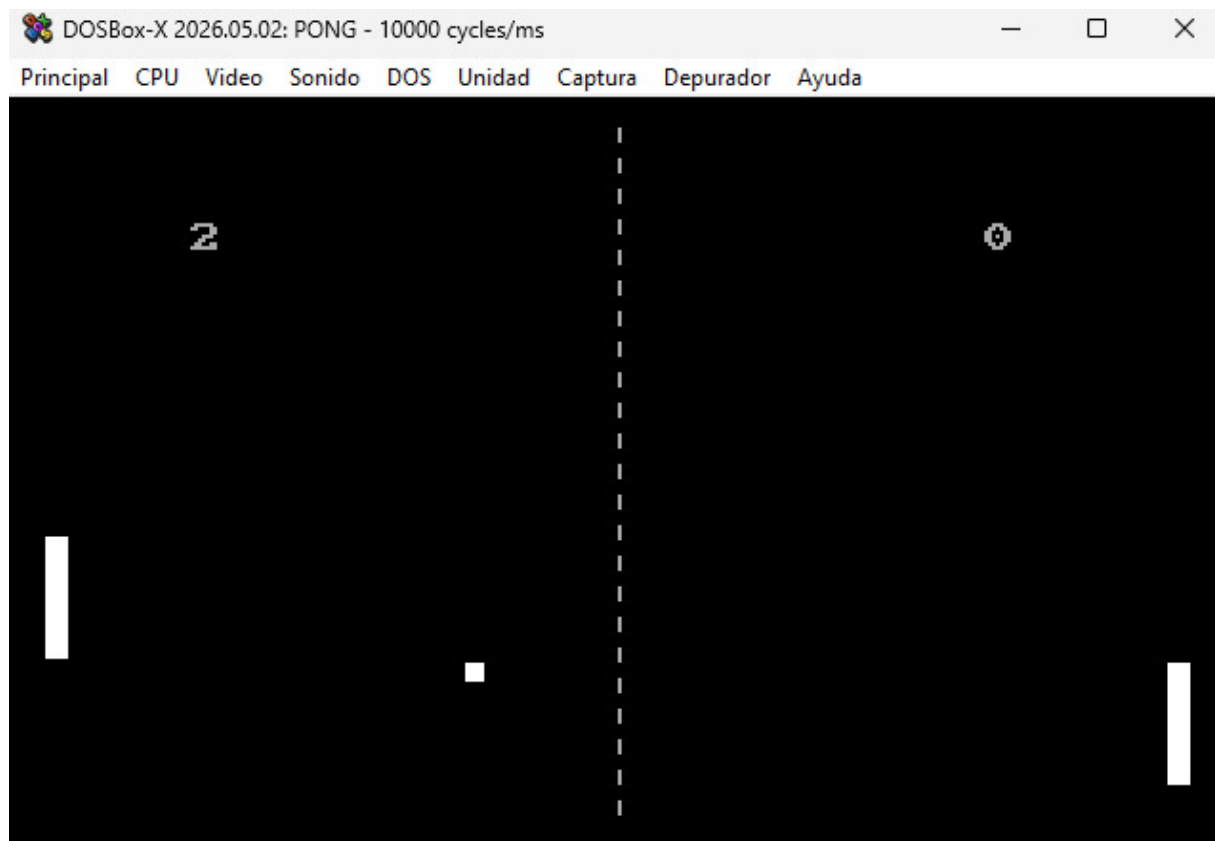


Figura 4: Aplicación en funcionamiento

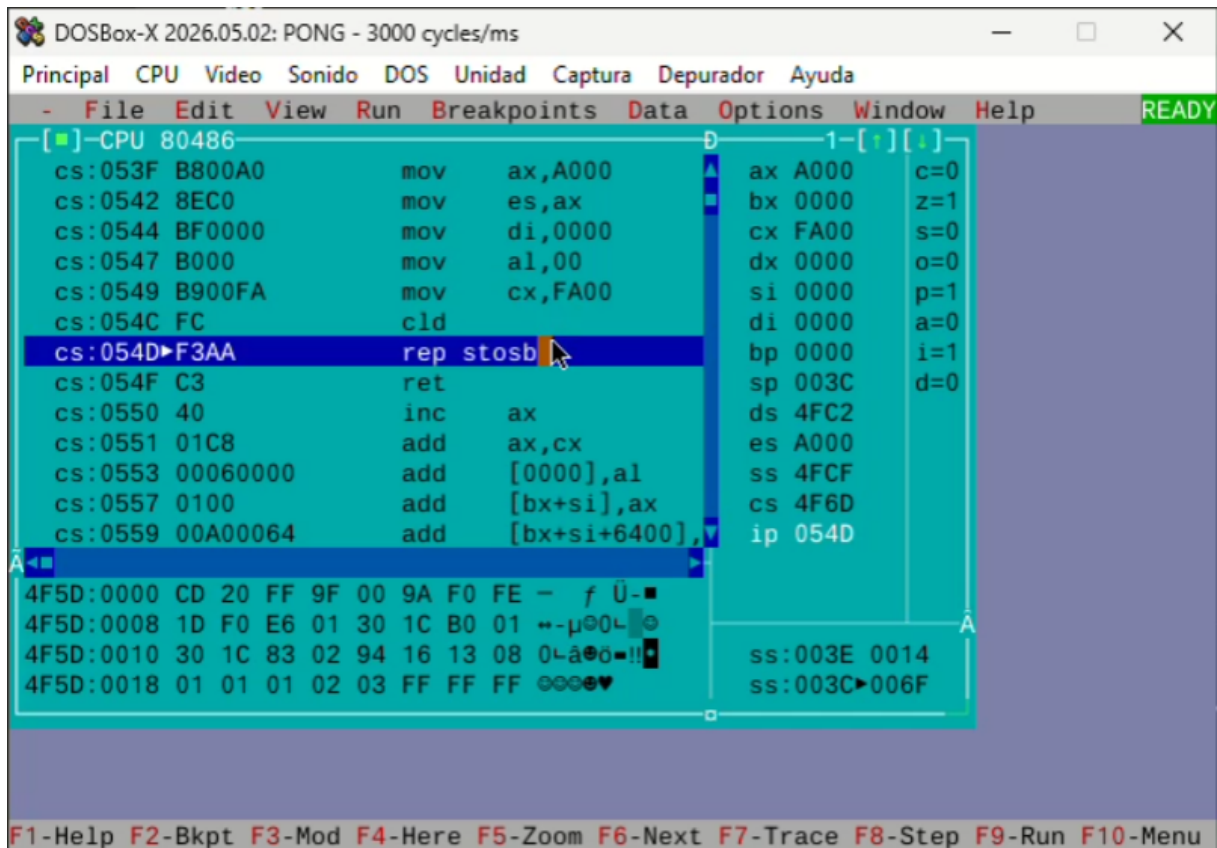


Figura 5: Cambio de registros con el juego en ejecución pt. 1

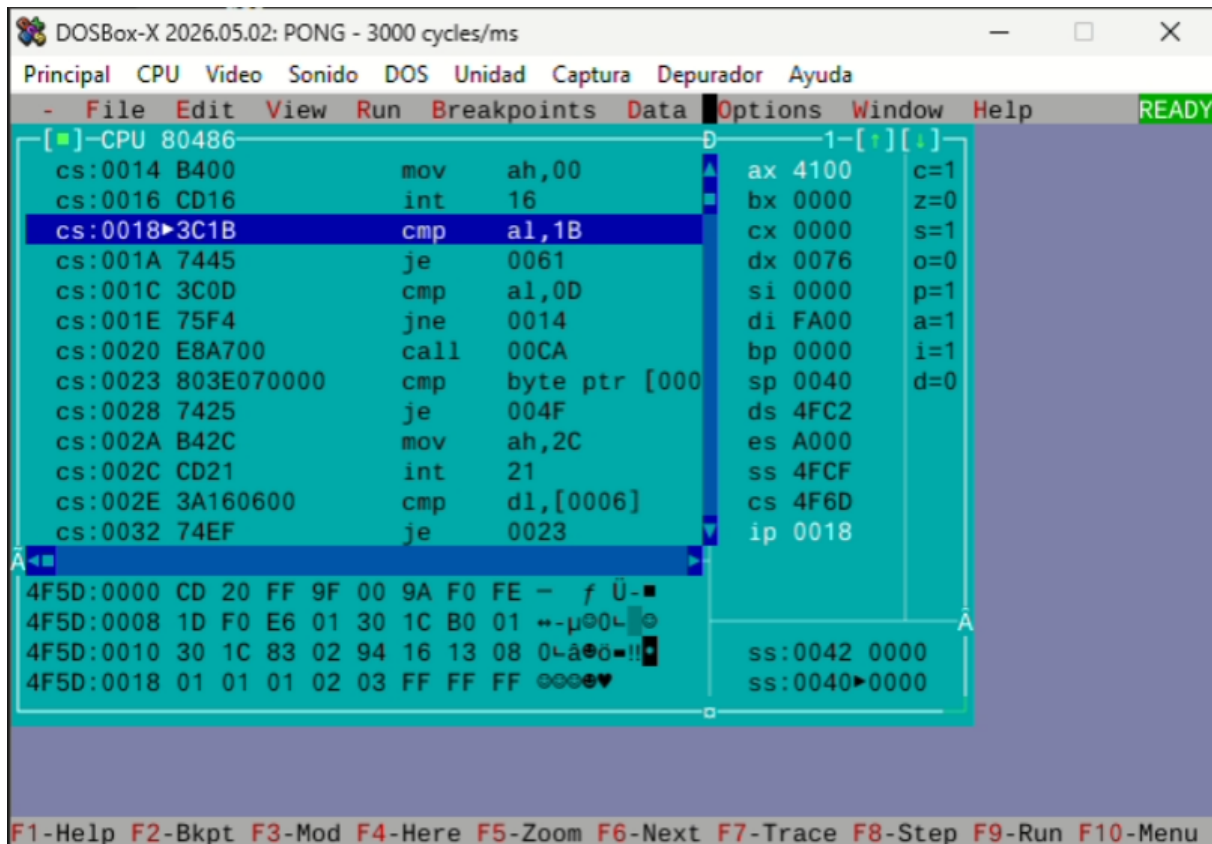


Figura 6: Cambio de registros con el juego en ejecución pt. 2

6. Discusiones y Mejoras Futuras

6.1 Dificultades Encontradas y Soluciones Implementadas

6.1.1. Sincronización del Ciclo de Juego

Uno de los primeros obstáculos fue lograr una velocidad de actualización consistente independientemente de la velocidad del procesador emulado. Una implementación ingenua con un simple lazo de retardo (*busy-wait* con LOOP) produce velocidades de juego radicalmente distintas según los ciclos configurados en DOSBox. La solución adoptada fue anclar el ciclo de juego al reloj del sistema operativo mediante INT 21h / función 2Ch, comparando la variable TIME_AUX con las centésimas de segundo actuales (DL). Este enfoque garantiza que el estado del juego se actualice como máximo 100 veces por segundo sin depender de la velocidad del CPU, aunque introduce la limitación de que la resolución temporal mínima queda fijada en 10 ms.

6.1.2. Detección de Colisiones con Signo en Registros de 16 Bits

El manejo de velocidades negativas para representar movimiento en sentido contrario presentó un desafío no trivial. Las variables BALL_VELOCITY_X y BALL_VELOCITY_Y se

declaran como palabras (DW) con signo, lo que implica que instrucciones como `CMP` interpretan correctamente los valores en complemento a dos, pero operaciones como `DIV` requieren que `DX` esté limpio (`XOR DX, DX`) para evitar resultados erróneos por extensión de signo. En el procedimiento `INCREASE_BALL_SPEED` fue necesario verificar el signo de la velocidad con `OR AX, AX` y ramificar el flujo para incrementar o decrementar según la dirección actual, evitando así que la aceleración invierta inadvertidamente el sentido del movimiento.

6.1.3. Control de Audio sin Bloqueo

Los procedimientos de sonido (`BEEP_PADDLE`, `BEEP_WALL`, `BEEP_SCORE`) implementan la duración del tono mediante bucles de espera activa (`LOOP`). Esto detiene completamente la ejecución del juego durante la reproducción del sonido, produciendo una pausa perceptible especialmente en `BEEP_SCORE`, que reproduce dos tonos consecutivos con silencio intermedio. La solución de compromiso fue ajustar empíricamente los valores de `CX` para cada procedimiento, minimizando la duración percibida sin eliminar el feedback auditivo. No obstante, este enfoque es inherentemente bloqueante y representa una limitación arquitectónica del diseño actual.

6.1.4. Gestión de la Paleta Derecha con Teclas de Función

Las teclas de flecha en el teclado PC generan códigos extendidos: al leerlas con `INT 16h` / servicio `00h`, el byte bajo (`AL`) retorna `00h` y el byte alto (`AH`) contiene el código de scan (`48h` para flecha arriba, `50h` para flecha abajo). Dado que el flujo del procedimiento `MOVE_PADDLES` lee la tecla una única vez, fue necesario estructurar el código de forma que la verificación de la paleta derecha opere sobre `AH` y no sobre `AL`, lo cual requirió cuidado especial para no sobrescribir `AH` antes de la comparación al procesar primero las teclas de la paleta izquierda.

6.1.5. Limitación de un Solo Evento de Teclado por Ciclo

La arquitectura de lectura de teclado implementada procesa como máximo una tecla por fotograma: si ambos jugadores presionan su tecla simultáneamente, solo se atiende la del Jugador 1. Esto se debe a que `INT 16h` / servicio `01h` detecta si hay alguna tecla en el buffer, y si la respuesta es afirmativa, se lee y procesa esa tecla antes de evaluar el movimiento de la paleta derecha. En la práctica este comportamiento es imperceptible a las velocidades de actualización del juego, pero constituye una simplificación consciente del diseño.

6.2 Análisis de Estabilidad del Producto

El programa muestra un comportamiento estable bajo las condiciones normales de operación en `DOSBox`. Sin embargo, se identificaron los siguientes aspectos que afectan la robustez del sistema:

- **Desbordamiento aritmético en posiciones:** Las variables de posición se declaran como palabras sin signo (DW), con un rango de 0 a 65.535. En condiciones nor-

males, las posiciones nunca abandonan el espacio de la pantalla (0–319 en X, 0–199 en Y), pero una velocidad inicial mal configurada podría causar un desbordamiento al sumar valores negativos a posiciones cercanas a cero, produciendo teletransportación de objetos al extremo opuesto de la pantalla. Las comprobaciones de límite implementadas mitigan este riesgo en la configuración actual de parámetros.

- **Dependencia del entorno DOS:** El programa utiliza INT 21h para servicios de tiempo y texto, e INT 10h y INT 16h para video y teclado. Estos vectores de interrupción son provistos por la BIOS y el sistema operativo DOS; fuera de DOSBox o de hardware real con DOS, el comportamiento no está garantizado.
- **Modo de video no restaurado ante interrupción abrupta:** Si el programa termina de forma anormal (por ejemplo, ante un error de división por cero o un salto a dirección inválida), el modo de video 13h permanece activo y el terminal queda inutilizable hasta que se restaure manualmente. El flujo normal sí restaura el modo texto (INT 10h / AL = 03h) antes de salir, pero no existe un manejador de excepciones que garantice esto ante fallos inesperados.
- **Buffer de teclado no vaciado al reiniciar:** Al transitar del estado de Game Over al reinicio de la partida, las teclas que el usuario pudo haber presionado durante la pantalla de espera permanecen en el buffer de INT 16h y son procesadas en los primeros fotogramas de la nueva partida, produciendo un movimiento inicial involuntario de las paletas.

6.3 Mejoras Futuras

6.3.1. Audio Asíncrono mediante Interrupción de Temporizador

La mejora más inmediata y de mayor impacto en la jugabilidad sería desvincular la reproducción de audio del hilo principal de ejecución. En la arquitectura 8086 esto es posible instalando un manejador personalizado para la interrupción del temporizador del sistema (INT 08h, que se activa 18,2 veces por segundo), de modo que el sonido se gestione en segundo plano sin pausar el ciclo de renderizado [6]. En procesadores modernos de 64 bits este problema se resuelve naturalmente con hilos del sistema operativo (*threads*), separando los contextos de audio, lógica de juego y renderizado.

6.3.2. Doble Buffer de Video para Eliminar el Parpadeo

El procedimiento actual borra la pantalla completa con `REP STOSB` y luego redibuja todos los objetos en cada fotograma. En hardware real esto produce un parpadeo visible (*flickering*) porque el monitor puede capturar el estado intermedio entre el borrado y el redibujo. La técnica estándar para eliminar este artefacto es el doble búfer (*double buffering*): preparar el fotograma completo en un buffer en memoria y copiarlo al framebuffer de video en una sola operación atómica, idealmente sincronizada con el retrazo vertical (*vertical retrace*) del monitor mediante el registro de estado de la VGA (puerto 3DAh) [6].

6.3.3. Inteligencia Artificial para Modo de un Jugador

Una extensión natural del proyecto sería añadir un modo de un jugador en el que la paleta derecha sea controlada por el sistema. Una implementación sencilla consistiría en un algoritmo de seguimiento proporcional: calcular la diferencia entre el centro de la paleta y la posición Y de la pelota, y mover la paleta en esa dirección a una velocidad máxima configurable. Esto introduce el concepto de agente reactivo implementado íntegramente en ensamblador, añadiendo variables de estado mínimas (dirección objetivo, velocidad del bot) al segmento de datos.

6.3.4. Escalabilidad a Arquitecturas de 32 y 64 Bits

En una arquitectura x86-64 el programa se beneficiaría de las siguientes mejoras estructurales [1]:

- **Registros extendidos (EAX, RAX):** Las posiciones y velocidades podrían almacenarse en registros de 32 bits, eliminando el riesgo de desbordamiento en cálculos intermedios y permitiendo resoluciones de pantalla significativamente mayores (Full HD 1920×1080 o superior) sin pérdida de precisión.
- **Instrucciones SIMD (SSE/AVX):** La detección de colisiones y la actualización de posiciones de múltiples objetos podrían paralelizarse usando registros vectoriales de 128 o 256 bits, procesando las coordenadas X e Y de varios objetos simultáneamente en una sola instrucción.
- **API gráfica de sistema operativo:** En un entorno moderno se reemplazaría el acceso directo al framebuffer por llamadas a una API gráfica (DirectX, OpenGL o Vulkan), delegando la sincronización vertical, la gestión de resolución y el doble búfer al driver de video, con lo que el código de aplicación se independiza del hardware concreto.
- **Modelo de memoria plano:** La arquitectura de 64 bits opera con un espacio de direcciones plano y lineal, eliminando completamente la complejidad del modelo segmentado del 8086 (registros de segmento, cálculo de dirección física, límite de 64 KB por segmento) y simplificando el acceso a estructuras de datos de mayor tamaño.

6.3.5. Persistencia de Puntuaciones y Tabla de Líderes

Actualmente el marcador se reinicia al terminar cada partida. Una mejora funcional sería almacenar las puntuaciones máximas en un archivo en disco utilizando los servicios de archivo de INT 21h (función 3Ch para crear, 40h para escribir y 3Fh para leer), añadiendo una pantalla de tabla de líderes entre la pantalla de Game Over y el menú principal.

7. Conclusiones y Referencias

7.1 Conclusiones

La implementación del videojuego Pong en lenguaje ensamblador Intel 8086 permitió comprender de manera concreta y verificable el funcionamiento de la jerarquía de memoria en una arquitectura real. A lo largo del desarrollo fue necesario operar simultáneamente con tres niveles de almacenamiento claramente diferenciados: los registros internos del procesador (AX, BX, CX, DX, SI, DI), la memoria principal direccionada a través de los segmentos de datos y pila, y la memoria de video mapeada en la dirección física A000:0000h. Cada nivel impuso restricciones propias: los registros ofrecen acceso en un solo ciclo de reloj pero en cantidad limitada, obligando a una administración cuidadosa para no sobrescribir valores intermedios entre instrucciones consecutivas; la memoria de datos permitió persistir el estado del juego entre llamadas a subrutinas, pero cada acceso introdujo ciclos adicionales de bus; y el framebuffer de video, aunque accesible como memoria ordinaria mediante `REP STOSB`, reveló la tensión entre ancho de banda de escritura y frecuencia de actualización visual. Esta experiencia materializó la pirámide de jerarquía de memoria descrita por Patterson y Hennessy [1]: a mayor cercanía al procesador, mayor velocidad y menor capacidad.

El ciclo de instrucción del 8086 (búsqueda, decodificación y ejecución) dejó de ser un concepto abstracto para convertirse en una restricción de diseño tangible. Cada decisión sobre el flujo de control tuvo que considerar la latencia de las interrupciones (INT 10h, INT 16h, INT 21h), el costo de los saltos condicionales que rompen la cola de prebúsqueda de 6 bytes del procesador, y la longitud de los bucles críticos como `CLEAR_SCREEN` y las rutinas de audio. La sincronización del ciclo de juego mediante el servicio de tiempo del DOS evidenció que un programa de tiempo real no puede ignorar cuánto tarda cada instrucción en completarse: un lazo de retardo basado en `LOOP` produce comportamientos distintos según la velocidad del procesador, mientras que anclar el estado al reloj del sistema desacopla la lógica de juego de la microarquitectura subyacente. Asimismo, la necesidad de preservar registros antes de cada `CALL` y restaurarlos tras el `RET` correspondiente hizo evidente el papel de la pila como estructura de soporte del ciclo de instrucción: sin ella, la anidación de subrutinas y el retorno correcto al punto de llamada serían imposibles.

Desde la perspectiva de la organización del computador, el proyecto demostró que un sistema de cómputo no es únicamente un procesador ejecutando instrucciones en aislamiento, sino un conjunto de subsistemas coordinados por buses, interrupciones y protocolos de temporización. El control del altavoz mediante escritura directa a los puertos 42h, 43h y 61h del temporizador 8253 ilustró cómo el bus de entrada/salida conecta al procesador con los periféricos, y cómo el software debe respetar la secuencia exacta de escrituras que el hardware espera para producir el comportamiento deseado. La gestión de las interrupciones de BIOS y DOS, por su parte, reveló el rol del vector de interrupciones como interfaz estándar entre el software de aplicación y el hardware: una capa de abstracción que, paradójicamente, solo se comprende en profundidad cuando se programa en el nivel más bajo. En definitiva, construir un videojuego funcional en ensamblador —con gráficos, sonido, entrada de usuario y lógica de estado— confirmó que los fundamentos de

la arquitectura de computadoras no son conocimientos históricos sino el sustrato sobre el cual descansa toda la pila de software moderna, y que su comprensión profunda es una ventaja técnica permanente para cualquier ingeniero de sistemas.

7.2 Referencias

Referencias

- [1] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5.^a ed. Waltham, MA, EE.UU.: Morgan Kaufmann, 2014.
- [2] K. R. Irvine, *Assembly Language for x86 Processors*, 7.^a ed. Upper Saddle River, NJ, EE.UU.: Pearson, 2014.
- [3] A. S. Tanenbaum y T. Austin, *Structured Computer Organization*, 6.^a ed. Upper Saddle River, NJ, EE.UU.: Pearson, 2013.
- [4] Intel Corporation, *8086 Family User's Manual*, Santa Clara, CA, EE.UU.: Intel Corporation, 1979. [En línea]. Disponible en: https://edge.edx.org/c4x/BITSPilani/EEE231/asset/8086_family_Users_Manual_1_.pdf
- [5] Intel Corporation, *8253 Programmable Interval Timer*, Ficha técnica, Santa Clara, CA, EE.UU.: Intel Corporation, 1979.
- [6] M. Abrash, *Michael Abrash's Graphics Programming Black Book*, Special ed. Scottsdale, AZ, EE.UU.: Coriolis Group, 1997.
- [7] C. Ericson, *Real-Time Collision Detection*. San Francisco, CA, EE.UU.: Morgan Kaufmann, 2005.
- [8] DOSBox Team, *DOSBox, An x86 Emulator with DOS*, versión 0.74-3, 2019. [En línea]. Disponible en: <https://www.dosbox.com>

A. Anexos

A.1 Código Fuente Completo (Documentado)

```
STACK SEGMENT PARA STACK          ; DEFINICION DEL SEGMENTO DE PILA
    DB 64 DUP(' ')                ; RESERVAR 64 BYTES PARA LA PILA
STACK ENDS                          ; FIN DEL SEGMENTO DE PILA

DATA SEGMENT PARA 'DATA'          ; DEFINICION DEL SEGMENTO DE DATOS

    WINDOW_WIDTH DW 140h           ; EL ANCHO DE LA VENTANA DE 320
    ; PIXELES EN HEXADECIMAL DADO POR EL VIDEO MODE
    WINDOW_HEIGHT DW 0C8h          ; EL ALTO DE LA VENTANA DE 200
    ; PIXELES EN HEXADECIMAL DADO POR EL VIDEO MODE
    WINDOW_BOUNDS DW 6             ; VARIABLE USADA PARA LAS
    ; COLISIONES

    TIME_AUX DB 0                  ; VARIABLE USADA PARA
    ; VERIFICAR SI EL TIEMPO HA CAMBIADO
    GAME_ACTIVE DB 1               ; INDICA SI EL JUEGO ESTA
    ; ACTIVO (1->TRUE 0-> FALSE)
    GAME_STARTED DB 0              ; INDICA SI EL JUEGO HA
    ; INICIADO (0 -> MENU, 1 -> JUGANDO)
    WINNER_INDEX DB 0              ; INDICA EL INDICE DEL
    ; GANADOR (1 -> PLAYER_ONE 2 -> PLAYER_TWO)

    BALL_X DW 0A0h                 ; POSICION EN X (COLUMNA)
    ; PARA LA PELOTA
    BALL_Y DW 64h                  ; POSICION EN Y (FILA)
    ; PARA LA PELOTA
    BALL_SIZE DW 04h               ; TAMA O DE LA PELOTA 4
    ; X4 PIXELES
    BALL_VELOCITY_X DW 05h          ; VELOCIDAD EN X (HORIZONTAL) DE
    ; LA PELOTA
    BALL_VELOCITY_Y DW 02h          ; VELOCIDAD EN Y (VERTICAL) DE LA
    ; PELOTA

    BALL_BASE_VEL_X DW 05h          ; VELOCIDAD BASE EN X (SE
    ; RESTAURA AL RESETEAR)
    BALL_BASE_VEL_Y DW 02h          ; VELOCIDAD BASE EN Y (SE
    ; RESTAURA AL RESETEAR)
    BALL_MAX_VEL_X DW 0Ch           ; VELOCIDAD MAXIMA EN X (12
    ; PIXELES)
    BALL_MAX_VEL_Y DW 07h           ; VELOCIDAD MAXIMA EN Y (7
    ; PIXELES)
    BOUNCE_COUNT DW 0000h           ; CONTADOR DE REBOTES PARA
    ; ACELERAR LA PELOTA

    BALL_ORIGINAL_POS_X DW 0A0h      ; POSICION DE LA PELOTA EN X (
    ; HORIZONTAL), CENTRO HORIZONTAL DE LA PANTALLA 160 EN HEXADECIMAL
    BALL_ORIGINAL_POS_Y DW 64h      ; POSICION DE LA PELOTA EN Y (
    ; VERTICAL), CENTRO VERTICAL DE LA PANTALLA 100 EN HEXADECIMAL

    PADDLE_LEFT_X DW 0Ah            ; POSICION ACTUAL DE LA PALETA
    ; IZQUIERDA EN X (HORIZONTAL)
    PADDLE_LEFT_Y DW 0Ah            ; POSICION ACTUAL DE LA PALETA
    ; IZQUIERDA EN Y (VERTICAL)

    PADDLE_RIGHT_X DW 130h          ; POSICION ACTUAL DE LA PALETA
    ; DERECHA EN X (HORIZONTAL)
    PADDLE_RIGHT_Y DW 0Ah           ; POSICION ACTUAL DE LA PALETA
    ; DERECHA EN Y (VERTICAL)

    PADDLE_WIDTH DW 05h             ; ANCHO DE LAS PALETAS
    PADDLE_HEIGHT DW 1Fh            ; ALTO DE LAS PALETAS
```

```

PADDLE_VELOCITY DW 05h ; VELOCIDAD DE LAS PALETAS

PLAYER_ONE_POINTS DB 0 ; PUNTOS DEL JUGADOR UNO (PALETA
IZQUIERDA)
PLAYER_TWO_POINTS DB 0 ; PUNTOS DEL JUGADOR DOS (PALETA
DERECHA)

PLAYER_ONE_POINT_TEXT DB '0', '$' ; TEXTO SOBRE LOS PUNTOS DEL JUGADOR UNO
PLAYER_TWO_POINT_TEXT DB '0', '$' ; TEXTO SOBRE LOS PUNTOS DEL JUGADOR DOS

TEXT_GAME_OVER_TITLE DB 'GAME_OVER', '$' ; TEXTO PARA EL TITULO DE LA
PANTALLA DE GAME OVER
TEXT_GAME_OVER_WINNER DB 'PLAYER_0_WIN', '$' ; TEXTO PARA MOSTRAR QUIEN ES EL
GANADOR EN LA PANTALLA DE GAME OVER

TEXT_MENU_TITLE DB 'PONG', '$'
TEXT_MENU_P1 DB 'PLAYER_1: UP/DOWN', '$'
TEXT_MENU_P2 DB 'PLAYER_2: UP/DOWN', '$'
TEXT_MENU_START DB 'PRESS_ENTER_TO_START', '$'
TEXT_MENU_WIN DB 'FIRST_TO_5_POINTS_WINS', '$'
TEXT_PLAY_AGAIN DB 'PRESS_ENTER_TO_PLAY_AGAIN', '$'
TEXT_PRESS_ESC DB 'PRESS_ESC_TO_EXIT', '$'

DATA ENDS ; FIN DEL SEGMENTO DE DATOS

CODE SEGMENT PARA 'CODE' ; DEFINICION DEL SEGMENTO DE CODIGO

MAIN PROC FAR ; PROCEDIMIENTO PRINCIPAL DEL
PROGRAMA
ASSUME CS:CODE, DS:DATA, SS:STACK ; ESTABLECER LOS SEGMENTOS A SUS DEBIDOS
REGISTROS (CS, DS, SS)
PUSH DS ; ENVIAR AL STACK EL
SEGMENTO DE DATOS (REGISTRO DS)
SUB AX, AX ; LIMPIAR EL REGISTRO
AX
PUSH AX ; ENVIAR EL REGISTRO AX
AL STACK
MOV AX, DATA ; GUARDAR EL CONTENIDO DEL DATA
SEGMENT EN EL REGISTRO AX
MOV DS, AX ; GUARDAR EN EL
REGISTRO DS EL CONTENIDO DEL REGISTRO AX
POP AX ; GUARDAR EL TOP DEL
STACK EN EL REGISTRO AX
POP AX ; GUARDAR EL TOP DEL
STACK EN EL REGISTRO AX

MOV AH, 00h ; PONER LA CONFIGURACION HACIA VIDEO MODE
MOV AL, 13h ; ELEGIR EL MODO DE VIDEO
INT 10h ; EJECUTAR INT

SHOW_MENU: ; ETIQUETA PARA MOSTRAR EL MENU PRINCIPAL
CALL DRAW_MENU_SCREEN ; DIBUJAR LA PANTALLA DEL MENU

WAIT_START_KEY: ; ETIQUETA PARA ESPERAR TECLA DE INICIO
MOV AH, 00h ; LEER TECLA
PRESIONADA
INT 16h ; EJECUTAR INT / LEER TECLA
CMP AL, 1Bh ; ESC -> SALIR
DEL PROGRAMA
JE EXIT_PROGRAM ; SI ES ESC, SALIR DEL PROGRAMA
CMP AL, 0Dh ; ENTER ->
INICIAR JUEGO
JNE WAIT_START_KEY ; SI NO ES ENTER, SEGUIR ESPERANDO

CALL RESET_GAME_STATE ; REINICIAR ESTADO DEL JUEGO

CHECK_TIME: ; ETIQUETA PARA VERIFICAR EL TIEMPO
CMP GAME_ACTIVE, 00h ; COMPARAR SI EL JUEGO ESTA ACTIVO

```

```
JE SHOW_GAME_OVER          ; SI NO ESTA ACTIVO, MOSTRAR GAME
OVER

MOV AH, 2Ch                 ; OBTENER EL TIEMPO DEL
SYSTEMA                     ;
INT 21h                    ; CH = HORA, CL =
MINUTO, DH = SEGUNDO, DL = 1/100 SEGUNDO

; VERIFICAR SI EL TIEMPO ACTUAL ES IGUAL AL ANTERIOR (TIME_AUX)
CMP DL, TIME_AUX           ; COMPARAR TIEMPO ACTUAL CON AUXILIAR

JE CHECK_TIME              ; SI ES IGUAL COMPARA
OTRA VEZ SINO REALIZAR EL RESTO DE ACCIONES
MOV TIME_AUX, DL           ; ACTUALIZAR VARIABLE DE TIEMPO
AUXILIAR

CALL CLEAR_SCREEN          ; LIMPIAR LA PANTALLA
PARA EVITAR LA VISUALIZACION DE PIXELES ANTERIORES

CALL MOVE_BALL             ; MOVER LA PELOTA
CALL DRAW_BALL             ; DIBUJAR LA PELOTA

CALL DRAW_CENTER_LINE      ; DIBUJAR LA LINEA CENTRAL
DIVISORIA

CALL MOVE_PADDLES          ; MOVER LAS PALETAS
MEDIANTE LOS INPUTS DEL TECLADO
CALL DRAW_PADDLES          ; DIBUJAR LAS PALETAS

CALL DRAW_UI              ; DIBUJAR INTERFAZ DE USUARIO (PUNTOS
)

JMP CHECK_TIME             ; LUEGO DE REALIZAR LAS
ACCIONES VOLVER A REVISAR EL TIEMPO

SHOW_GAME_OVER:           ; ETIQUETA PARA MOSTRAR PANTALLA DE
GAME OVER
CALL DRAW_GAME_OVER_SCREEN ; DIBUJAR PANTALLA DE GAME
OVER

; DRAW_GAME_OVER_SCREEN YA ESPERA UNA TECLA
; AL RETORNAR, AL TIENE LA TECLA PRESIONADA
CMP AL, 0Dh                ; ENTER -> JUGAR
DE NUEVO
JE RESTART_GAME           ; SI ES ENTER, REINICIAR JUEGO
CMP AL, 1Bh                ; ESC -> SALIR
JE EXIT_PROGRAM           ; SI ES ESC, SALIR DEL PROGRAMA
JMP SHOW_GAME_OVER        ; OTRA TECLA -> ESPERAR
DE NUEVO

RESTART_GAME:             ; ETIQUETA PARA REINICIAR EL
JUEGO
CALL RESET_GAME_STATE     ; REINICIAR VARIABLES DEL
JUEGO
JMP CHECK_TIME            ; VOLVER AL LOOP
PRINCIPAL

EXIT_PROGRAM:             ; ETIQUETA PARA SALIR DEL PROGRAMA
; RESTAURAR MODO DE VIDEO A TEXTO ANTES DE SALIR
MOV AH, 00h               ; PREPARAR PARA RESTAURAR MODO DE
VIDEO
MOV AL, 03h               ; SELECCIONAR MODO TEXTO
INT 10h                   ; EJECUTAR INT / CAMBIAR MODO DE
VIDEO
MOV AH, 4Ch               ; PREPARAR PARA FINALIZAR PROGRAMA
INT 21h                   ; EJECUTAR INT / SALIR DEL PROGRAMA

RET                        ; RETORNAR AL
SISTEMA OPERATIVO
```



```
MAIN ENDP ; FIN DEL
PROCEDIMIENTO PRINCIPAL

DRAW_MENU_SCREEN PROC NEAR ; PROCEDIMIENTO PARA DIBUJAR EL MENU
PRINCIPAL
CALL CLEAR_SCREEN ; LIMPIAR PANTALLA

; TITULO "PONG" - CENTRADO
MOV AH, 02h ; PREPARAR PARA POSICIONAR CURSOR
MOV BH, 00h ; SELECCIONAR PAGINA DE VIDEO 0
MOV DH, 03h ; FILA 3
MOV DL, 13h ; COLUMNA 19 (
CENTRADO EN 40 COLS)
INT 10h ; EJECUTAR INT / POSICIONAR CURSOR
MOV AH, 09h ; PREPARAR PARA MOSTRAR CADENA
LEA DX, TEXT_MENU_TITLE ; CARGAR DIRECCION DEL TITULO
INT 21h ; EJECUTAR INT / MOSTRAR CADENA

; LINEA SEPARADORA DE GUIONES (MANUAL PIXEL A PIXEL CON TEXTO)
; CONTROLES JUGADOR 1
MOV AH, 02h ; PREPARAR PARA POSICIONAR CURSOR
MOV BH, 00h ; SELECCIONAR PAGINA DE VIDEO 0
MOV DH, 06h ; FILA 6
MOV DL, 0Dh ; COLUMNA 13
INT 10h ; EJECUTAR INT / POSICIONAR CURSOR
MOV AH, 09h ; PREPARAR PARA MOSTRAR CADENA
LEA DX, TEXT_MENU_P1 ; CARGAR DIRECCION DEL TEXTO CONTROLES P1
INT 21h ; EJECUTAR INT / MOSTRAR CADENA

; CONTROLES JUGADOR 2
MOV AH, 02h ; PREPARAR PARA POSICIONAR CURSOR
MOV BH, 00h ; SELECCIONAR PAGINA DE VIDEO 0
MOV DH, 08h ; FILA 8
MOV DL, 0Bh ; COLUMNA 11
INT 10h ; EJECUTAR INT / POSICIONAR CURSOR
MOV AH, 09h ; PREPARAR PARA MOSTRAR CADENA
LEA DX, TEXT_MENU_P2 ; CARGAR DIRECCION DEL TEXTO CONTROLES P2
INT 21h ; EJECUTAR INT / MOSTRAR CADENA

; CONDICION DE VICTORIA
MOV AH, 02h ; PREPARAR PARA POSICIONAR CURSOR
MOV BH, 00h ; SELECCIONAR PAGINA DE VIDEO 0
MOV DH, 0Bh ; FILA 11
MOV DL, 09h ; COLUMNA 9
INT 10h ; EJECUTAR INT / POSICIONAR CURSOR
MOV AH, 09h ; PREPARAR PARA MOSTRAR CADENA
LEA DX, TEXT_MENU_WIN ; CARGAR DIRECCION DEL TEXTO CONDICION
VICTORIA
INT 21h ; EJECUTAR INT / MOSTRAR CADENA

; MENSAJE DE INICIO
MOV AH, 02h ; PREPARAR PARA POSICIONAR CURSOR
MOV BH, 00h ; SELECCIONAR PAGINA DE VIDEO 0
MOV DH, 0Eh ; FILA 14
MOV DL, 0Ah ; COLUMNA 10
INT 10h ; EJECUTAR INT / POSICIONAR CURSOR
MOV AH, 09h ; PREPARAR PARA MOSTRAR CADENA
LEA DX, TEXT_MENU_START ; CARGAR DIRECCION DEL MENSAJE DE INICIO
INT 21h ; EJECUTAR INT / MOSTRAR CADENA

RET ; RETORNAR DEL PROCEDIMIENTO
DRAW_MENU_SCREEN ENDP ; FIN DEL PROCEDIMIENTO

RESET_GAME_STATE PROC NEAR ; PROCEDIMIENTO PARA REINICIAR ESTADO DEL
JUEGO
; RESETEAR PUNTOS
MOV PLAYER_ONE_POINTS, 00h ; PONER PUNTOS JUGADOR 1 A 0
MOV PLAYER_TWO_POINTS, 00h ; PONER PUNTOS JUGADOR 2 A 0
CALL UPDATE_TEXT_PLAYER_ONE_POINTS ; ACTUALIZAR TEXTO PUNTOS J1
```

```

CALL UPDATE_TEXT_PLAYER_TWO_POINTS ; ACTUALIZAR TEXTO PUNTOS J2

; RESETEAR POSICIONES DE PALETAS
MOV PADDLE_LEFT_Y, 0Ah ; RESTAURAR POSICION Y PALETA IZQUIERDA
MOV PADDLE_RIGHT_Y, 0Ah ; RESTAURAR POSICION Y PALETA DERECHA

; RESETEAR PELOTA
CALL RESET_BALL_POSITION ; RESTAURAR POSICION DE LA PELOTA

; RESETEAR VELOCIDADES DE LA PELOTA A LOS VALORES BASE
MOV AX, BALL_BASE_VEL_X ; CARGAR VELOCIDAD BASE EN X
MOV BALL_VELOCITY_X, AX ; RESTAURAR VELOCIDAD X
MOV AX, BALL_BASE_VEL_Y ; CARGAR VELOCIDAD BASE EN Y
MOV BALL_VELOCITY_Y, AX ; RESTAURAR VELOCIDAD Y

; RESETEAR CONTADOR DE REBOTES
MOV BOUNCE_COUNT, 0000h ; REINICIAR CONTADOR DE REBOTES

; RESETEAR INDICE DEL GANADOR
MOV WINNER_INDEX, 00h ; LIMPIAR INDICE DEL GANADOR

; ACTIVAR EL JUEGO
MOV GAME_ACTIVE, 01h ; ESTABLECER JUEGO COMO ACTIVO

RET ; RETORNAR DEL PROCEDIMIENTO
RESET_GAME_STATE ENDP ; FIN DEL PROCEDIMIENTO
DRAW_CENTER_LINE PROC NEAR ; PROCEDIMIENTO PARA DIBUJAR LINEA CENTRAL
MOV DX, 08h ; EMPEZAR DESDE Y
=8 (DEJAR BORDE SUPERIOR)
MOV CX, 0A0h ; X = 160 (CENTRO
HORIZONTAL)

DRAW_CENTER_LOOP: ; ETIQUETA DEL BUCLE PARA DIBUJAR
PUNTEADO
; DIBUJAR UN SEGMENTO DE 4 PÍXELES DE ALTO
MOV AH, 0Ch ; PREPARAR PARA DIBUJAR PIXEL
MOV AL, 07h ; COLOR GRIS
OSCURO (NO TAN BRILLANTE COMO LAS PALETAS)
MOV BH, 00h ; SELECCIONAR PAGINA DE VIDEO
INT 10h ; EJECUTAR INT / DIBUJAR PIXEL
INC DX ; AVANZAR A SIGUIENTE FILA
MOV AH, 0Ch ; PREPARAR PARA DIBUJAR PIXEL
MOV AL, 07h ; COLOR GRIS OSCURO
MOV BH, 00h ; SELECCIONAR PAGINA DE VIDEO
INT 10h ; EJECUTAR INT / DIBUJAR PIXEL
INC DX ; AVANZAR A SIGUIENTE FILA
MOV AH, 0Ch ; PREPARAR PARA DIBUJAR PIXEL
MOV AL, 07h ; COLOR GRIS OSCURO
MOV BH, 00h ; SELECCIONAR PAGINA DE VIDEO
INT 10h ; EJECUTAR INT / DIBUJAR PIXEL
INC DX ; AVANZAR A SIGUIENTE FILA
MOV AH, 0Ch ; PREPARAR PARA DIBUJAR PIXEL
MOV AL, 07h ; COLOR GRIS OSCURO
MOV BH, 00h ; SELECCIONAR PAGINA DE VIDEO
INT 10h ; EJECUTAR INT / DIBUJAR PIXEL

ADD DX, 05h ; SALTAR 5
PÍXELES (ESPACIO ENTRE SEGMENTOS -> EFECTO PUNTEADO)

MOV AX, WINDOW_HEIGHT ; CARGAR ALTURA DE VENTANA
SUB AX, 08h ; DEJAR BORDE INFERIOR
CMP DX, AX ; COMPARAR POSICION Y CON LIMITE INFERIOR
JL DRAW_CENTER_LOOP ; CONTINUAR SI NO LLEGAMOS AL
BORDE

RET ; RETORNAR DEL
PROCEDIMIENTO
DRAW_CENTER_LINE ENDP ; FIN DEL PROCEDIMIENTO

```

```

DRAW_BALL PROC NEAR                                ; PROCEDIMIENTO PARA DIBUJAR LA PELOTA

    MOV CX, BALL_X                                ; SELECCIONAR CORDENADA X (
        COLUMNNA) INICIAL
    MOV DX, BALL_Y                                ; SELECCIONAR COORDENADA Y (FILA)
        INICIAL

    DRAW_BALL_LABEL:                               ; ETIQUETA PARA DIBUJAR LA PELOTA
        PIXEL POR PIXEL

        MOV AH, 0Ch                                ; DIBUJAR UN PIXEL EN MODO
            GRAFICO
        MOV AL, 0Fh                                ; PONER DE COLOR BLANCO AL PIXEL
        MOV BH, 00h                                ; SELECCIONAR PAGINA DE VIDEO
        INT 10h                                    ; EJECUTAR INT
        INC CX                                    ; CX = CX + 1

        ; CX - BALL_X > BALL_SIZE (Y -> IR A LA SIGUIENTE LINEA, N ->
            SEGUIR A LA SIGUIENTE COLUMNA)
        MOV AX, CX                                ; COPIAR POSICION X ACTUAL A AX
        SUB AX, BALL_X                             ; RESTAR POSICION INICIAL X
        CMP AX, BALL_SIZE                          ; COMPARAR CON TAMA O DE PELOTA

        JNG DRAW_BALL_LABEL                        ; SALTAR SI NO ES MAYOR (SEGUIR EN MISMA
            FILA)

        MOV CX, BALL_X                             ; PONER EL REGISTRO CX EN LA
            COLUMNA INICIAL
        INC DX                                    ; AVANZAR UNA LINEA

        ; DX - BALL_Y > BALL_SIZE (Y -> TERMINAR PROCEDIMIENTO, N ->
            SEGUIR A LA SIGUIENTE LINEA)
        MOV AX, DX                                ; COPIAR POSICION Y ACTUAL A AX
        SUB AX, BALL_Y                             ; RESTAR POSICION INICIAL Y
        CMP AX, BALL_SIZE                          ; COMPARAR CON TAMA O DE PELOTA
        JNG DRAW_BALL_LABEL                        ; SALTAR SI NO ES MAYOR (SEGUIR DIBUJANDO
            )

    RET                                            ;

    RETORNAR DEL PROCEDIMIENTO

DRAW_BALL ENDP                                    ; FIN DEL
    PROCEDIMIENTO PARA DIBUJAR LA PELOTA

DRAW_PADDLES PROC NEAR                            ; PROCEDIMIENTO
    PARA DIBUJAR LAS PALETAS
    MOV CX, PADDLE_LEFT_X                          ; SELECCIONAR
        CORDENADA X (COLUMNA) INICIAL
    MOV DX, PADDLE_LEFT_Y                          ; SELECCIONAR
        COORDENADA Y (FILA) INICIAL

    DRAW_PADDLE_LEFT_HORIZONTAL:                   ; ETIQUETA PARA DIBUJAR
        LA PALETA IZQUIERDA
        MOV AH, 0Ch                                ; DIBUJAR
            UN PIXEL EN MODO GRAFICO
        MOV AL, 0Fh                                ; PONER
            DE COLOR BLANCO AL PIXEL
        MOV BH, 00h                                ;
            SELECCIONAR PAGINA DE VIDEO
        INT 10h                                    ;
            EJECUTAR INT

        INC CX                                    ;
            CX = CX + 1
        ; CX - PADDLE_LEFT_X > PADDLE_WIDTH (Y -> IR A LA SIGUIENTE LINEA
            , N -> SEGUIR A LA SIGUIENTE COLUMNA)
        MOV AX, CX                                ; COPIAR POSICION X ACTUAL A
            AX
        SUB AX, PADDLE_LEFT_X                      ; RESTAR POSICION INICIAL X

```

```
CMP AX, PADDLE_WIDTH          ; COMPARAR CON ANCHO DE
PALETA

JNG DRAW_PADDLE_LEFT_HORIZONTAL ; SALTAR SI NO ES MAYOR (
SEGUIR EN MISMA FILA)

MOV CX, PADDLE_LEFT_X          ; PONER EL
REGISTRO CX EN LA COLUMNA INICIAL
INC DX                          ;
AVANZAR UNA LINEA

; DX - PADDLE_LEFT_Y > PADDLE_HEIGHT (Y -> TERMINAR PROCEDIMIENTO
, N -> SEGUIR A LA SIGUIENTE LINEA)
MOV AX, DX                      ; COPIAR POSICION Y ACTUAL A
AX
SUB AX, PADDLE_LEFT_Y          ; RESTAR POSICION INICIAL Y
CMP AX, PADDLE_HEIGHT          ; COMPARAR CON ALTO DE PALETA
JNG DRAW_PADDLE_LEFT_HORIZONTAL ; SALTAR SI NO ES MAYOR (
SEGUIR DIBUJANDO)

MOV CX, PADDLE_RIGHT_X         ; SELECCIONAR
CORDENADA X (COLUMNA) INICIAL
MOV DX, PADDLE_RIGHT_Y         ; SELECCIONAR
COORDENADA Y (FILA) INICIAL

DRAW_PADDLE_RIGHT_HORIZONTAL:  ; ETIQUETA PARA DIBUJAR
LA PALETA DERECHA
MOV AH, 0Ch                    ; DIBUJAR
UN PIXEL EN MODO GRAFICO
MOV AL, 0Fh                    ; PONER
DE COLOR BLANCO AL PIXEL
MOV BH, 00h                    ;
SELECCIONAR PAGINA DE VIDEO
INT 10h                        ;
EJECUTAR INT

INC CX                          ;
CX = CX + 1
; CX - PADDLE_RIGHT_X > PADDLE_WIDTH (Y -> IR A LA SIGUIENTE
LINEA, N -> SEGUIR A LA SIGUIENTE COLUMNA)
MOV AX, CX                      ; COPIAR POSICION X ACTUAL A
AX
SUB AX, PADDLE_RIGHT_X         ; RESTAR POSICION INICIAL X
CMP AX, PADDLE_WIDTH           ; COMPARAR CON ANCHO DE
PALETA

JNG DRAW_PADDLE_RIGHT_HORIZONTAL ; SALTAR SI NO ES MAYOR (
SEGUIR EN MISMA FILA)

MOV CX, PADDLE_RIGHT_X         ; PONER EL
REGISTRO CX EN LA COLUMNA INICIAL
INC DX                          ;
AVANZAR UNA LINEA

; DX - PADDLE_RIGHT_Y > PADDLE_HEIGHT (Y -> TERMINAR
PROCEDIMIENTO, N -> SEGUIR A LA SIGUIENTE LINEA)
MOV AX, DX                      ; COPIAR POSICION Y ACTUAL A
AX
SUB AX, PADDLE_RIGHT_Y         ; RESTAR POSICION INICIAL Y
CMP AX, PADDLE_HEIGHT          ; COMPARAR CON ALTO DE PALETA
JNG DRAW_PADDLE_RIGHT_HORIZONTAL ; SALTAR SI NO ES MAYOR (
SEGUIR DIBUJANDO)

RET                             ;
RETORNAR DEL PROCEDIMIENTO
DRAW_PADDLES ENDP              ; FIN DEL
PROCEDIMIENTO PARA DIBUJAR LAS PALETAS
```

```

MOVE_BALL PROC NEAR                                ; PROCEDIMIENTO
    PARA MOVER LA PELOTA

    MOV AX, BALL_VELOCITY_X                        ; MOVER LA PELOTA HORIZONTALMENTE
    ADD BALL_X, AX                                ; SUMAR VELOCIDAD X A POSICION X

    MOV AX, WINDOW_BOUNDS                          ; CARGAR LIMITE DE VENTANA
    CMP BALL_X, AX                                ; BALL_X < WINDOW_BOUNDS
    (Y -> COLISIONO)

    JG CONTINUE_X1                                ; SI ES MAYOR, SEGUIR VERIFICANDO
    JMP GIVE_POINT_TO_PLAYER_TWO                  ; SUMAR UN PUNTO AL JUGADOR DOS Y RESETAR
    LA PELOTA
CONTINUE_X1:                                       ; ETIQUETA DE CONTINUACION X1

    MOV AX, WINDOW_WIDTH                          ; CARGAR ANCHO DE VENTANA
    SUB AX, BALL_SIZE                              ; RESTAR TAMA O DE PELOTA
    SUB AX, WINDOW_BOUNDS                          ; RESTAR LIMITE DE VENTANA
    CMP BALL_X, AX                                ; BALL_X > WINDOW_WIDTH -
    BALL_SIZE - WINDOW_BOUNDS (Y -> COLISIONO)
    JL CONTINUE_X2                                ; SI ES MENOR, SEGUIR VERIFICANDO
    JMP GIVE_POINT_TO_PLAYER_ONE                  ; SUMAR UN PUNTO AL JUGADOR UNO Y RESETAR
    LA PELOTA
CONTINUE_X2:                                       ; ETIQUETA DE CONTINUACION X2

    MOV AX, BALL_VELOCITY_Y                        ; MOVER LA PELOTA VERTICALMENTE
    ADD BALL_Y, AX                                ; SUMAR VELOCIDAD Y A POSICION Y

    MOV AX, WINDOW_BOUNDS                          ; CARGAR LIMITE DE VENTANA
    CMP BALL_Y, AX                                ; BALL_Y < WINDOW_BOUNDS
    (Y -> COLISIONO)

    JG CONTINUE_Y1                                ; SI ES MAYOR, SEGUIR VERIFICANDO
    JMP NEG_VELOCITY_Y                            ; INVERTIR VELOCIDAD EN Y
CONTINUE_Y1:                                       ; ETIQUETA DE CONTINUACION Y1

    MOV AX, WINDOW_HEIGHT                          ; CARGAR ALTURA DE VENTANA
    SUB AX, BALL_SIZE                              ; RESTAR TAMA O DE PELOTA
    SUB AX, WINDOW_BOUNDS                          ; RESTAR LIMITE DE VENTANA
    CMP BALL_Y, AX                                ; COMPARAR POSICION Y CON LIMITE INFERIOR
    JL CONTINUE_Y2                                ; BALL_Y > WINDOW_HEIGHT
    - BALL_SIZE - WINDOW_BOUNDS (Y -> COLISIONO)
    JMP NEG_VELOCITY_Y                            ; INVERTIR VELOCIDAD EN Y
CONTINUE_Y2:                                       ; ETIQUETA DE CONTINUACION Y2

    ; VERIFICAR COLISION CON PALETA DERECHA
    MOV AX, BALL_X                                ; CARGAR POSICION X DE PELOTA
    ADD AX, BALL_SIZE                              ; SUMAR TAMA O DE PELOTA
    CMP AX, PADDLE_RIGHT_X                        ; COMPARAR CON POSICION X DE PALETA
    DERECHA
    JNG CHECK_COLLISION_WITH_PADDLE_LEFT          ; SI NO HAY COLISION, VERIFICAR
    PALETA IZQUIERDA

    MOV AX, PADDLE_RIGHT_X                        ; CARGAR POSICION X DE PALETA DERECHA
    ADD AX, PADDLE_WIDTH                          ; SUMAR ANCHO DE PALETA
    CMP BALL_X, AX                                ; COMPARAR CON POSICION X DE PELOTA
    JNL CHECK_COLLISION_WITH_PADDLE_LEFT          ; SI NO HAY COLISION, VERIFICAR
    PALETA IZQUIERDA

    MOV AX, BALL_Y                                ; CARGAR POSICION Y DE PELOTA
    ADD AX, BALL_SIZE                              ; SUMAR TAMA O DE PELOTA
    CMP AX, PADDLE_RIGHT_Y                        ; COMPARAR CON POSICION Y DE PALETA
    DERECHA
    JNG CHECK_COLLISION_WITH_PADDLE_LEFT          ; SI NO HAY COLISION, VERIFICAR
    PALETA IZQUIERDA

    MOV AX, PADDLE_RIGHT_Y                        ; CARGAR POSICION Y DE PALETA DERECHA
    ADD AX, PADDLE_HEIGHT                          ; SUMAR ALTO DE PALETA
    CMP BALL_Y, AX                                ; COMPARAR CON POSICION Y DE PELOTA
    JNL CHECK_COLLISION_WITH_PADDLE_LEFT          ; SI NO HAY COLISION, VERIFICAR
    PALETA IZQUIERDA

```

```
; COLISION CON PALETA DERECHA CONFIRMADA
NEG BALL_VELOCITY_X          ; INVERTIR VELOCIDAD HORIZONTAL
CALL INCREASE_BALL_SPEED     ; AUMENTAR VELOCIDAD PROGRESIVAMENTE
CALL BEEP_PADDLE             ; SONIDO DE REBOTE EN
    PALETA
RET                           ; RETORNAR DEL PROCEDIMIENTO

CHECK_COLLISION_WITH_PADDLE_LEFT:    ; ETIQUETA PARA VERIFICAR
    COLISION CON PALETA IZQUIERDA
    MOV AX, BALL_X                ; CARGAR POSICION X DE
        PELOTA
    ADD AX, BALL_SIZE              ; SUMAR TAMA O DE PELOTA
    CMP AX, PADDLE_LEFT_X         ; COMPARAR CON POSICION X
        DE PALETA IZQUIERDA
    JNG MOVE_BALL_EXIT            ; SI NO HAY COLISION,
        SALIR

    MOV AX, PADDLE_LEFT_X         ; CARGAR POSICION X DE
        PALETA IZQUIERDA
    ADD AX, PADDLE_WIDTH          ; SUMAR ANCHO DE PALETA
    CMP BALL_X, AX               ; COMPARAR CON POSICION X
        DE PELOTA
    JNL MOVE_BALL_EXIT            ; SI NO HAY COLISION,
        SALIR

    MOV AX, BALL_Y                ; CARGAR POSICION Y DE
        PELOTA
    ADD AX, BALL_SIZE              ; SUMAR TAMA O DE PELOTA
    CMP AX, PADDLE_LEFT_Y         ; COMPARAR CON POSICION Y
        DE PALETA IZQUIERDA
    JNG MOVE_BALL_EXIT            ; SI NO HAY COLISION,
        SALIR

    MOV AX, PADDLE_LEFT_Y         ; CARGAR POSICION Y DE
        PALETA IZQUIERDA
    ADD AX, PADDLE_HEIGHT         ; SUMAR ALTO DE PALETA
    CMP BALL_Y, AX               ; COMPARAR CON POSICION Y
        DE PELOTA
    JNL MOVE_BALL_EXIT            ; SI NO HAY COLISION,
        SALIR

; COLISION CON PALETA IZQUIERDA CONFIRMADA
NEG BALL_VELOCITY_X          ; INVERTIR VELOCIDAD HORIZONTAL
CALL INCREASE_BALL_SPEED     ; AUMENTAR VELOCIDAD
    PROGRESIVAMENTE
CALL BEEP_PADDLE             ; SONIDO DE
    REBOTE EN PALETA

MOVE_BALL_EXIT:                ; ETIQUETA DE SALIDA DEL
    MOVIMIENTO
    RET                           ;
        RETORNAR DEL PROCEDIMIENTO

GIVE_POINT_TO_PLAYER_ONE:      ; ETIQUETA PARA
    RESETEAR LA POSICION DE LA PELOTA Y A ADIR UN PUNTO AL JUGADOR UNO
    INC PLAYER_ONE_POINTS      ; INCREMENTAR PUNTOS DEL
        JUGADOR UNO
    CALL RESET_BALL_POSITION    ; RESTAURAR POSICION DE
        LA PELOTA
    CALL RESET_BALL_SPEED       ; RESETEAR VELOCIDAD AL
        MARCAR PUNTO
    CALL BEEP_SCORE             ; SONIDO DE PUNTO
        ANOTADO
    CALL UPDATE_TEXT_PLAYER_ONE_POINTS ; ACTUALIZAR TEXTO DE PUNTOS
        J1
    CMP PLAYER_ONE_POINTS, 05h  ; COMPARAR SI LLEGO A 5
        PUNTOS
```

```

        JGE GAME_OVER                ; SI ES MAYOR O IGUAL,
        TERMINAR JUEGO
    RET                                ;
        RETORNAR DESPUES DE RESETEAR POSICION

GIVE_POINT_TO_PLAYER_TWO:            ; ETIQUETA PARA DAR PUNTO AL
    JUGADOR DOS
    INC PLAYER_TWO_POINTS            ; INCREMENTAR PUNTOS DEL
    JUGADOR DOS
    CALL RESET_BALL_POSITION         ; RESTAURAR POSICION DE
    LA PELOTA
    CALL RESET_BALL_SPEED            ; RESETEAR VELOCIDAD AL
    MARCAR PUNTO
    CALL BEEP_SCORE                  ; SONIDO DE PUNTO
    ANOTADO
    CALL UPDATE_TEXT_PLAYER_TWO_POINTS ; ACTUALIZAR TEXTO DE PUNTOS
    J2
    CMP PLAYER_TWO_POINTS, 05h       ; COMPARAR SI LLEGO A 5
    PUNTOS
    JGE GAME_OVER                    ; SI ES MAYOR O IGUAL,
    TERMINAR JUEGO
    RET                                ;
        RETORNAR DESPUES DE RESETEAR POSICION

NEG_VELOCITY_Y:                      ;
    ETIQUETA PARA INVERTIR LA VELOCIDAD EN Y
    NEG BALL_VELOCITY_Y             ; NEGAR VELOCIDAD
    VERTICAL (CAMBIAR DIRECCION)
    CALL BEEP_WALL                   ; SONIDO DE
    REBOTE EN PARED
    RET                                ;
        RETORNAR DESPUES DE INVERTIR VELOCIDAD

GAME_OVER:                           ; LLAMAR
    CUANDO SE HALLAN COMPLETADO LOS 5 PUNTOS
    CMP PLAYER_ONE_POINTS, 05h       ; COMPARAR PUNTOS DEL JUGADOR UNO
    JNL WINNER_IS_PLAYER_ONE        ; SI ES MAYOR O IGUAL, GANA
    JUGADOR UNO
    JMP WINNER_IS_PLAYER_TWO         ; SI NO, GANA JUGADOR DOS

    WINNER_IS_PLAYER_ONE:            ; ETIQUETA CUANDO GANA JUGADOR
    UNO
        MOV WINNER_INDEX, 01h        ; ESTABLECER INDICE DE
        GANADOR A 1
        JMP CONTINUE_GAME_OVER      ; SALTAR A CONTINUACION

    WINNER_IS_PLAYER_TWO:            ; ETIQUETA CUANDO GANA JUGADOR
    DOS
        MOV WINNER_INDEX, 02h        ; ESTABLECER INDICE DE
        GANADOR A 2
        JMP CONTINUE_GAME_OVER      ; SALTAR A CONTINUACION

    CONTINUE_GAME_OVER:              ; ETIQUETA DE CONTINUACION DE
    GAME OVER
    MOV GAME_ACTIVE, 00h             ; DESACTIVAR EL JUEGO
    RET                                ; RETORNAR

MOVE_BALL ENDP                      ; FIN DEL
    PROCEDIMIENTO PARA MOVER LA PELOTA

INCREASE_BALL_SPEED PROC NEAR        ; PROCEDIMIENTO PARA AUMENTAR VELOCIDAD
    INC BOUNCE_COUNT                 ; INCREMENTAR CONTADOR DE REBOTES

    ; SOLO AUMENTAR CADA 3 REBOTES
    MOV AX, BOUNCE_COUNT              ; CARGAR CONTADOR DE REBOTES
    MOV BX, 0003h                     ; DIVISOR = 3
    XOR DX, DX                         ; LIMPIAR DX PARA DIVISION
    DIV BX                             ; DIVIDIR CONTADOR ENTRE 3

```

```
CMP DX, 0000h ; VERIFICAR SI RESTO ES 0
JNE SPEED_EXIT ; SI NO ES
MULTIPLO DE 3, NO AUMENTAR

; AUMENTAR VELOCIDAD EN X (MANTENIENDO EL SIGNO)
MOV AX, BALL_VELOCITY_X ; CARGAR VELOCIDAD ACTUAL EN X
OR AX, AX ; VERIFICAR SIGNO DE VELOCIDAD X
JNS INC_VEL_X_POSITIVE ; SI ES POSITIVO SALTAR

; VELOCIDAD X NEGATIVA: DECREMENTAR (HACER MAS NEGATIVA)
CMP AX, -0Ch ; COMPARAR CON
MAXIMO NEGATIVO (-12)
JLE SKIP_INC_X ; SI YA LLEGO AL
MAXIMO NEGATIVO, NO AUMENTAR
DEC AX ; DECREMENTAR (HACER MAS NEGATIVO)
MOV BALL_VELOCITY_X, AX ; GUARDAR NUEVA VELOCIDAD X
JMP CHECK_VEL_Y ; SALTAR A VERIFICAR VELOCIDAD Y

INC_VEL_X_POSITIVE: ; ETIQUETA PARA VELOCIDAD X POSITIVA
; VELOCIDAD X POSITIVA
MOV BX, BALL_MAX_VEL_X ; CARGAR VELOCIDAD MAXIMA EN X
CMP AX, BX ; COMPARAR VELOCIDAD ACTUAL CON
MAXIMA
JGE SKIP_INC_X ; SI YA LLEGO AL
MAXIMO, NO AUMENTAR
INC AX ; INCREMENTAR VELOCIDAD
MOV BALL_VELOCITY_X, AX ; GUARDAR NUEVA VELOCIDAD X

SKIP_INC_X: ; ETIQUETA PARA SALTAR INCREMENTO X

CHECK_VEL_Y: ; ETIQUETA PARA VERIFICAR VELOCIDAD Y
; AUMENTAR VELOCIDAD EN Y (MANTENIENDO EL SIGNO)
MOV AX, BALL_VELOCITY_Y ; CARGAR VELOCIDAD ACTUAL EN Y
OR AX, AX ; VERIFICAR SIGNO DE VELOCIDAD Y
JNS INC_VEL_Y_POSITIVE ; SI ES POSITIVO SALTAR

; VELOCIDAD Y NEGATIVA
CMP AX, -07h ; MAXIMO NEGATIVO
EN Y (-7)
JLE SPEED_EXIT ; SI YA LLEGO AL MAXIMO, SALIR
DEC AX ; DECREMENTAR (HACER MAS NEGATIVO)
MOV BALL_VELOCITY_Y, AX ; GUARDAR NUEVA VELOCIDAD Y
JMP SPEED_EXIT ; SALTAR A SALIDA

INC_VEL_Y_POSITIVE: ; ETIQUETA PARA VELOCIDAD Y POSITIVA
MOV BX, BALL_MAX_VEL_Y ; CARGAR VELOCIDAD MAXIMA EN Y
CMP AX, BX ; COMPARAR VELOCIDAD ACTUAL CON
MAXIMA
JGE SPEED_EXIT ; SI YA LLEGO AL MAXIMO, SALIR
INC AX ; INCREMENTAR VELOCIDAD
MOV BALL_VELOCITY_Y, AX ; GUARDAR NUEVA VELOCIDAD Y

SPEED_EXIT: ; ETIQUETA DE SALIDA
RET ; RETORNAR DEL PROCEDIMIENTO
INCREASE_BALL_SPEED ENDP ; FIN DEL PROCEDIMIENTO

RESET_BALL_SPEED PROC NEAR ; PROCEDIMIENTO PARA RESETEAR VELOCIDAD
; RESTAURAR VELOCIDADES BASE (PRESERVAR SIGNO ACTUAL DE X E Y)
MOV AX, BALL_VELOCITY_X ; CARGAR VELOCIDAD ACTUAL EN X
OR AX, AX ; VERIFICAR SIGNO
JNS RESET_VEL_X_POS ; SI ES POSITIVO, SALTAR

; VELOCIDAD X ERA NEGATIVA -> RESTAURAR BASE NEGATIVA
MOV AX, BALL_BASE_VEL_X ; CARGAR VELOCIDAD BASE EN X
NEG AX ; NEGAR PARA HACERLA NEGATIVA
MOV BALL_VELOCITY_X, AX ; GUARDAR VELOCIDAD RESTAURADA
JMP CHECK_RESET_Y ; SALTAR A VERIFICAR Y

RESET_VEL_X_POS: ; ETIQUETA PARA VELOCIDAD X POSITIVA
```



```

        MOV AX, BALL_BASE_VEL_X          ; CARGAR VELOCIDAD BASE EN X
        MOV BALL_VELOCITY_X, AX          ; GUARDAR VELOCIDAD RESTAURADA

CHECK_RESET_Y:                          ; ETIQUETA PARA VERIFICAR Y
MOV AX, BALL_VELOCITY_Y                ; CARGAR VELOCIDAD ACTUAL EN Y
OR AX, AX                              ; VERIFICAR SIGNO
JNS RESET_VEL_Y_POS                    ; SI ES POSITIVO, SALTAR

MOV AX, BALL_BASE_VEL_Y                ; CARGAR VELOCIDAD BASE EN Y
NEG AX                                  ; NEGAR PARA HACERLA NEGATIVA
MOV BALL_VELOCITY_Y, AX                ; GUARDAR VELOCIDAD RESTAURADA
JMP RESET_SPEED_EXIT                  ; SALTAR A SALIDA

RESET_VEL_Y_POS:                        ; ETIQUETA PARA VELOCIDAD Y POSITIVA
MOV AX, BALL_BASE_VEL_Y                ; CARGAR VELOCIDAD BASE EN Y
MOV BALL_VELOCITY_Y, AX                ; GUARDAR VELOCIDAD RESTAURADA

RESET_SPEED_EXIT:                       ; ETIQUETA DE SALIDA
MOV BOUNCE_COUNT, 0000h                ; REINICIAR CONTADOR DE
REBOTES
RET                                     ; RETORNAR DEL PROCEDIMIENTO
RESET_BALL_SPEED ENDP                  ; FIN DEL PROCEDIMIENTO

BEEP_PADDLE PROC NEAR                  ; PROCEDIMIENTO PARA SONIDO DE PALETA
; FRECUENCIA ~800 Hz: DIVISOR = 1193180 / 800 = 1491 = 05D3h
MOV AL, 0B6h                          ; CONFIGURAR TIMER 2: MODO 3 (
CUADRADA), CANAL 2
OUT 43h, AL                            ; ENVIAR COMANDO DE CONFIGURACION AL TIMER
MOV AX, 05D3h                          ; DIVISOR PARA ~800 Hz
OUT 42h, AL                            ; ENVIAR BYTE BAJO DEL
DIVISOR
MOV AL, AH                             ; CARGAR BYTE ALTO DEL DIVISOR
OUT 42h, AL                            ; ENVIAR BYTE ALTO DEL
DIVISOR

IN AL, 61h                             ; LEER PUERTO DEL ALTAVOZ
OR AL, 03h                             ; ACTIVAR BITS 0 Y 1 (
ENCENDER ALTAVOZ)
OUT 61h, AL                            ; ENVIAR DATOS AL PUERTO DEL ALTAVOZ

; DURACION CORTA: LOOP DE ESPERA ~15ms
MOV CX, 0C000h                        ; CARGAR CONTADOR PARA DURACION DEL TONO
BEEP_P_WAIT: LOOP BEEP_P_WAIT          ; BUCLE DE ESPERA

IN AL, 61h                             ; LEER PUERTO DEL ALTAVOZ
AND AL, 0FCh                          ; DESACTIVAR BITS 0 Y 1 (
APAGAR ALTAVOZ)
OUT 61h, AL                            ; ENVIAR DATOS AL PUERTO DEL ALTAVOZ
RET                                     ; RETORNAR DEL PROCEDIMIENTO
BEEP_PADDLE ENDP                       ; FIN DEL PROCEDIMIENTO

BEEP_WALL PROC NEAR                   ; PROCEDIMIENTO PARA SONIDO DE PARED
; FRECUENCIA ~400 Hz: DIVISOR = 1193180 / 400 = 2982 = 0BA6h

MOV AL, 0B6h                          ; CONFIGURAR TIMER 2: MODO 3, CANAL 2
OUT 43h, AL                            ; ENVIAR COMANDO DE CONFIGURACION AL
TIMER
MOV AX, 0BA6h                          ; DIVISOR PARA ~400 Hz
OUT 42h, AL                            ; ENVIAR BYTE BAJO DEL DIVISOR
MOV AL, AH                             ; CARGAR BYTE ALTO DEL DIVISOR
OUT 42h, AL                            ; ENVIAR BYTE ALTO DEL DIVISOR

IN AL, 61h                             ; LEER PUERTO DEL ALTAVOZ
OR AL, 03h                             ; ACTIVAR BITS 0 Y 1 (ENCENDER ALTAVOZ)
OUT 61h, AL                            ; ENVIAR DATOS AL PUERTO DEL ALTAVOZ

MOV CX, 08000h                        ; CARGAR CONTADOR PARA DURACION DEL TONO
BEEP_W_WAIT: LOOP BEEP_W_WAIT          ; BUCLE DE ESPERA

```

```

        IN AL, 61h                ; LEER PUERTO DEL ALTAVOZ
        AND AL, 0FCh              ; DESACTIVAR BITS 0 Y 1 (APAGAR ALTAVOZ)
        OUT 61h, AL               ; ENVIAR DATOS AL PUERTO DEL ALTAVOZ
        RET                       ; RETORNAR DEL PROCEDIMIENTO
BEEP_WALL ENDP                   ; FIN DEL PROCEDIMIENTO

BEEP_SCORE PROC NEAR                ; PROCEDIMIENTO PARA SONIDO DE PUNTO
; PRIMER TONO: ~600 Hz
MOV AL, 0B6h                      ; CONFIGURAR TIMER 2: MODO 3, CANAL 2
OUT 43h, AL                       ; ENVIAR COMANDO DE CONFIGURACION AL TIMER
MOV AX, 07D0h                     ; DIVISOR PARA ~600 Hz
OUT 42h, AL                       ; ENVIAR BYTE BAJO DEL DIVISOR
MOV AL, AH                        ; CARGAR BYTE ALTO DEL DIVISOR
OUT 42h, AL                       ; ENVIAR BYTE ALTO DEL DIVISOR

IN AL, 61h                        ; LEER PUERTO DEL ALTAVOZ
OR AL, 03h                       ; ACTIVAR BITS 0 Y 1 (ENCENDER ALTAVOZ)
OUT 61h, AL                       ; ENVIAR DATOS AL PUERTO DEL ALTAVOZ

MOV CX, 0FFFFh                   ; CARGAR CONTADOR PARA DURACION DEL TONO
BEEP_S1_WAIT: LOOP BEEP_S1_WAIT ; BUCLE DE ESPERA

IN AL, 61h                        ; LEER PUERTO DEL ALTAVOZ
AND AL, 0FCh                     ; DESACTIVAR BITS 0 Y 1 (APAGAR ALTAVOZ)
OUT 61h, AL                       ; ENVIAR DATOS AL PUERTO DEL ALTAVOZ

; BREVE SILENCIO
MOV CX, 4000h                    ; CARGAR CONTADOR PARA SILENCIO
BEEP_S_SIL: LOOP BEEP_S_SIL ; BUCLE DE ESPERA (SILENCIO)

; SEGUNDO TONO: ~1000 Hz (MAS AGUDO)
MOV AL, 0B6h                      ; CONFIGURAR TIMER 2: MODO 3, CANAL 2
OUT 43h, AL                       ; ENVIAR COMANDO DE CONFIGURACION AL TIMER
MOV AX, 04B0h                     ; DIVISOR PARA ~1000 Hz
OUT 42h, AL                       ; ENVIAR BYTE BAJO DEL DIVISOR
MOV AL, AH                        ; CARGAR BYTE ALTO DEL DIVISOR
OUT 42h, AL                       ; ENVIAR BYTE ALTO DEL DIVISOR

IN AL, 61h                        ; LEER PUERTO DEL ALTAVOZ
OR AL, 03h                       ; ACTIVAR BITS 0 Y 1 (ENCENDER ALTAVOZ)
OUT 61h, AL                       ; ENVIAR DATOS AL PUERTO DEL ALTAVOZ

MOV CX, 0FFFFh                   ; CARGAR CONTADOR PARA DURACION DEL TONO
BEEP_S2_WAIT: LOOP BEEP_S2_WAIT ; BUCLE DE ESPERA

IN AL, 61h                        ; LEER PUERTO DEL ALTAVOZ
AND AL, 0FCh                     ; DESACTIVAR BITS 0 Y 1 (APAGAR ALTAVOZ)
OUT 61h, AL                       ; ENVIAR DATOS AL PUERTO DEL ALTAVOZ
RET                               ; RETORNAR DEL PROCEDIMIENTO
BEEP_SCORE ENDP                   ; FIN DEL PROCEDIMIENTO

MOVE_PADDLES PROC NEAR                ; PROCEDIMIENTO PARA MOVER
LAS PALETAS
MOV AH, 01h                      ; VERIFICAR SI HAY TECLA PRESIONADA
INT 16h                          ; EJECUTAR INT / VERIFICAR TECLA
JZ CHECK_RIGHT_PADDLE_MOVEMENT ; ZF = 1 NO HAY TECLA PRECIONADA

MOV AH, 00h                      ; REVISAR
; QUE TECLA ESTA SIENDO PRECIONADA
INT 16h                          ; EJECUTAR INT / LEER TECLA

; SI ES "W" o "w" MOVER HACIA ARRIBA
CMP AL, 77h                      ; "w"
JE MOVE_LEFT_PADDLE_UP           ; SI ES "w", MOVER PALETA
; IZQUIERDA ARRIBA
CMP AL, 57h                      ; "W"
JE MOVE_LEFT_PADDLE_UP           ; SI ES "W", MOVER PALETA
; IZQUIERDA ARRIBA

```

```
; SI ES "S" o "s" MOVER HACIA ABAJO
CMP AL, 73h                                     ;
"s"
JE MOVE_LEFT_PADDLE_DOWN                      ; SI ES "
s", MOVER PALETA IZQUIERDA ABAJO
CMP AL, 53h                                     ;
"S"
JE MOVE_LEFT_PADDLE_DOWN                      ; SI ES "
S", MOVER PALETA IZQUIERDA ABAJO

JMP CHECK_RIGHT_PADDLE_MOVEMENT               ; SALTAR A VERIFICAR
PALETA DERECHA

MOVE_LEFT_PADDLE_UP:                          ; ETIQUETA PARA MOVER
PALETA IZQUIERDA ARRIBA
MOV AX, PADDLE_VELOCITY                       ; CARGAR
VELOCIDAD DE PALETA
SUB PADDLE_LEFT_Y, AX                         ; RESTAR
VELOCIDAD A POSICION Y

MOV AX, WINDOW_BOUNDS                        ; CARGAR LIMITE
DE VENTANA
CMP PADDLE_LEFT_Y, AX                        ; COMPARAR
POSICION CON LIMITE
JL FIX_PADDLE_LEFT_TOP_POSITION              ; SI ES MENOR,
CORREGIR POSICION
JMP CHECK_RIGHT_PADDLE_MOVEMENT              ; SALTAR A
VERIFICAR PALETA DERECHA

FIX_PADDLE_LEFT_TOP_POSITION:                 ; ETIQUETA PARA
CORREGIR POSICION TOPE
MOV AX, WINDOW_BOUNDS                        ; CARGAR LIMITE
DE VENTANA
MOV PADDLE_LEFT_Y, AX                       ; FIJAR POSICION
EN LIMITE
JMP CHECK_RIGHT_PADDLE_MOVEMENT              ; SALTAR A
VERIFICAR PALETA DERECHA

MOVE_LEFT_PADDLE_DOWN:                       ; ETIQUETA PARA MOVER
PALETA IZQUIERDA ABAJO
MOV AX, PADDLE_VELOCITY                       ; CARGAR
VELOCIDAD DE PALETA
ADD PADDLE_LEFT_Y, AX                        ; SUMAR VELOCIDAD
A POSICION Y

MOV AX, WINDOW_HEIGHT                        ; CARGAR ALTURA
DE VENTANA
SUB AX, WINDOW_BOUNDS                        ; RESTAR LIMITE
DE VENTANA
SUB AX, PADDLE_HEIGHT                        ; RESTAR ALTO DE
PALETA
CMP PADDLE_LEFT_Y, AX                        ; COMPARAR
POSICION CON LIMITE INFERIOR
JG FIX_PADDLE_LEFT_BOTTOM_POSITION           ; SI ES MAYOR, CORREGIR
POSICION
JMP CHECK_RIGHT_PADDLE_MOVEMENT              ; SALTAR A
VERIFICAR PALETA DERECHA

FIX_PADDLE_LEFT_BOTTOM_POSITION:              ; ETIQUETA PARA
CORREGIR POSICION BASE
MOV PADDLE_LEFT_Y, AX                       ; FIJAR POSICION
EN LIMITE INFERIOR
JMP CHECK_RIGHT_PADDLE_MOVEMENT              ; SALTAR A
VERIFICAR PALETA DERECHA

CHECK_RIGHT_PADDLE_MOVEMENT:                 ; ETIQUETA PARA VERIFICAR
MOVIMIENTO PALETA DERECHA
```

```

    CMP AH, 48h ;
    FLECHA ARRIBA
    JE MOVE_RIGHT_PADDLE_UP ; SI ES
    FLECHA ARRIBA, MOVER ARRIBA

    CMP AH, 50h ;
    FLECHA ABAJO
    JE MOVE_RIGHT_PADDLE_DOWN ; SI ES
    FLECHA ABAJO, MOVER ABAJO
    JMP EXIT_PADDLE_MOVEMENT ; SALTAR A SALIDA

MOVE_RIGHT_PADDLE_UP: ; ETIQUETA PARA
    MOVER PALETA DERECHA ARRIBA
    MOV AX, PADDLE_VELOCITY ; CARGAR
    VELOCIDAD DE PALETA
    SUB PADDLE_RIGHT_Y, AX ; RESTAR
    VELOCIDAD A POSICION Y

    MOV AX, WINDOW_BOUNDS ; CARGAR LIMITE
    DE VENTANA
    CMP PADDLE_RIGHT_Y, AX ; COMPARAR
    POSICION CON LIMITE
    JL FIX_PADDLE_RIGHT_TOP_POSITION ; SI ES MENOR,
    CORREGIR POSICION
    JMP EXIT_PADDLE_MOVEMENT ; SALTAR A SALIDA

FIX_PADDLE_RIGHT_TOP_POSITION: ; ETIQUETA PARA
    CORREGIR POSICION TOPE
    MOV AX, WINDOW_BOUNDS ; CARGAR
    LIMITE DE VENTANA
    MOV PADDLE_RIGHT_Y, AX ; FIJAR
    POSICION EN LIMITE
    JMP EXIT_PADDLE_MOVEMENT ; SALTAR A SALIDA

MOVE_RIGHT_PADDLE_DOWN: ; ETIQUETA PARA
    MOVER PALETA DERECHA ABAJO
    MOV AX, PADDLE_VELOCITY ; CARGAR
    VELOCIDAD DE PALETA
    ADD PADDLE_RIGHT_Y, AX ; SUMAR VELOCIDAD
    A POSICION Y

    MOV AX, WINDOW_HEIGHT ; CARGAR ALTURA
    DE VENTANA
    SUB AX, WINDOW_BOUNDS ; RESTAR LIMITE
    DE VENTANA
    SUB AX, PADDLE_HEIGHT ; RESTAR ALTO DE
    PALETA
    CMP PADDLE_RIGHT_Y, AX ; COMPARAR
    POSICION CON LIMITE INFERIOR
    JG FIX_PADDLE_RIGHT_BOTTOM_POSITION ; SI ES MAYOR,
    CORREGIR POSICION
    JMP EXIT_PADDLE_MOVEMENT ; SALTAR A SALIDA

FIX_PADDLE_RIGHT_BOTTOM_POSITION: ; ETIQUETA PARA
    CORREGIR POSICION BASE
    MOV PADDLE_RIGHT_Y, AX ; FIJAR
    POSICION EN LIMITE INFERIOR
    JMP EXIT_PADDLE_MOVEMENT ; SALTAR A SALIDA

EXIT_PADDLE_MOVEMENT: ; ETIQUETA DE SALIDA
    RET ; RETORNAR DEL PROCEDIMIENTO
MOVE_PADDLES ENDP ; FIN DEL PROCEDIMIENTO

RESET_BALL_POSITION PROC NEAR ; PROCEDIMIENTO PARA RESETEAR POSICION DE
PELOTA
    MOV AX, BALL_ORIGINAL_POS_X ; CARGAR POSICION ORIGINAL EN X
    MOV BALL_X, AX ; RESTAURAR POSICION X DE PELOTA

    MOV AX, BALL_ORIGINAL_POS_Y ; CARGAR POSICION ORIGINAL EN Y

```

```

MOV BALL_Y, AX ; RESTAURAR POSICION Y DE PELOTA

RET ; RETORNAR DEL PROCEDIMIENTO
RESET_BALL_POSITION ENDP ; FIN DEL PROCEDIMIENTO

DRAW_UI PROC NEAR ; PROCEDIMIENTO PARA DIBUJAR INTERFAZ DE
USUARIO

; DIBUJAR LOS PUNTOS DEL JUGADOR UNO
MOV AH, 02h ; PREPARAR PARA POSICIONAR CURSOR
MOV BH, 00h ; SELECCIONAR PAGINA DE VIDEO 0
MOV DH, 04h ; FILA 4
MOV DL, 06h ; COLUMNA 6
INT 10h ; EJECUTAR INT / POSICIONAR CURSOR

MOV AH, 09h ; PREPARAR PARA MOSTRAR CADENA
LEA DX, PLAYER_ONE_POINT_TEXT ; CARGAR DIRECCION DEL TEXTO DE PUNTOS J1
INT 21h ; EJECUTAR INT / MOSTRAR CADENA

; DIBUJAR LOS PUNTOS DEL JUGADOR DOS
MOV AH, 02h ; PREPARAR PARA POSICIONAR CURSOR
MOV BH, 00h ; SELECCIONAR PAGINA DE VIDEO 0
MOV DH, 04h ; FILA 4
MOV DL, 20h ; COLUMNA 32
INT 10h ; EJECUTAR INT / POSICIONAR CURSOR

MOV AH, 09h ; PREPARAR PARA MOSTRAR CADENA
LEA DX, PLAYER_TWO_POINT_TEXT ; CARGAR DIRECCION DEL TEXTO DE PUNTOS J2
INT 21h ; EJECUTAR INT / MOSTRAR CADENA

RET ; RETORNAR DEL PROCEDIMIENTO
DRAW_UI ENDP ; FIN DEL PROCEDIMIENTO

UPDATE_TEXT_PLAYER_ONE_POINTS PROC NEAR ; PROCEDIMIENTO PARA ACTUALIZAR TEXTO DE
PUNTOS J1
XOR AX, AX ; LIMPIAR REGISTRO AX
MOV AL, PLAYER_ONE_POINTS ; CARGAR PUNTOS DEL JUGADOR UNO
ADD AL, 30h ; CONVERTIR A CARACTER ASCII
MOV [PLAYER_ONE_POINT_TEXT], AL ; GUARDAR EN VARIABLE DE TEXTO
RET ; RETORNAR DEL PROCEDIMIENTO
UPDATE_TEXT_PLAYER_ONE_POINTS ENDP ; FIN DEL PROCEDIMIENTO

UPDATE_TEXT_PLAYER_TWO_POINTS PROC NEAR ; PROCEDIMIENTO PARA ACTUALIZAR TEXTO DE
PUNTOS J2
XOR AX, AX ; LIMPIAR REGISTRO AX
MOV AL, PLAYER_TWO_POINTS ; CARGAR PUNTOS DEL JUGADOR DOS
ADD AL, 30h ; CONVERTIR A CARACTER ASCII
MOV [PLAYER_TWO_POINT_TEXT], AL ; GUARDAR EN VARIABLE DE TEXTO
RET ; RETORNAR DEL PROCEDIMIENTO
UPDATE_TEXT_PLAYER_TWO_POINTS ENDP ; FIN DEL PROCEDIMIENTO

DRAW_GAME_OVER_SCREEN PROC NEAR ; PROCEDIMIENTO PARA DIBUJAR GAME OVER

CALL CLEAR_SCREEN ; LIMPIAR PANTALLA
; MOSTRAR EL TITULO DEL MENU
MOV AH, 02h ; PREPARAR PARA POSICIONAR CURSOR
MOV BH, 00h ; SELECCIONAR PAGINA DE VIDEO 0
MOV DH, 04h ; FILA 4
MOV DL, 04h ; COLUMNA 4
INT 10h ; EJECUTAR INT / POSICIONAR CURSOR

MOV AH, 09h ; PREPARAR PARA MOSTRAR CADENA
LEA DX, TEXT_GAME_OVER_TITLE ; CARGAR DIRECCION DEL TITULO GAME
OVER
INT 21h ; EJECUTAR INT / MOSTRAR CADENA

; MOSTRAR EL GANADOR
MOV AH, 02h ; PREPARAR PARA POSICIONAR CURSOR
MOV BH, 00h ; SELECCIONAR PAGINA DE VIDEO 0

```

```
MOV DH, 06h ; FILA 6
MOV DL, 04h ; COLUMNA 4
INT 10h ; EJECUTAR INT / POSICIONAR CURSOR

CALL UPDATE_WINNER_TEXT ; ACTUALIZAR TEXTO DEL GANADOR

MOV AH, 09h ; PREPARAR PARA MOSTRAR CADENA
LEA DX, TEXT_GAME_OVER_WINNER ; CARGAR DIRECCION DEL TEXTO GANADOR
INT 21h ; EJECUTAR INT / MOSTRAR CADENA

MOV AH, 02h ; PREPARAR PARA POSICIONAR CURSOR
MOV BH, 00h ; SELECCIONAR PAGINA DE VIDEO 0
MOV DH, 09h ; FILA 9
MOV DL, 04h ; COLUMNA 4
INT 10h ; EJECUTAR INT / POSICIONAR CURSOR
MOV AH, 09h ; PREPARAR PARA MOSTRAR CADENA
LEA DX, TEXT_PLAY_AGAIN ; CARGAR DIRECCION DEL TEXTO JUGAR DE
NUEVO
INT 21h ; EJECUTAR INT / MOSTRAR CADENA

MOV AH, 02h ; PREPARAR PARA POSICIONAR CURSOR
MOV BH, 00h ; SELECCIONAR PAGINA DE VIDEO 0
MOV DH, 0Bh ; FILA 11
MOV DL, 04h ; COLUMNA 4
INT 10h ; EJECUTAR INT / POSICIONAR CURSOR
MOV AH, 09h ; PREPARAR PARA MOSTRAR CADENA
LEA DX, TEXT_PRESS_ESC ; CARGAR DIRECCION DEL TEXTO SALIR
CON ESC
INT 21h ; EJECUTAR INT / MOSTRAR CADENA

; ESPERAR UNA PULSACION Y RETORNARLA EN AL
MOV AH, 00h ; PREPARAR PARA LEER TECLA
INT 16h ; AL = CARACTER
ASCII DE LA TECLA

RET ; RETORNAR DEL PROCEDIMIENTO
DRAW_GAME_OVER_SCREEN ENDP ; FIN DEL PROCEDIMIENTO

UPDATE_WINNER_TEXT PROC NEAR ; PROCEDIMIENTO PARA ACTUALIZAR TEXTO DEL
GANADOR
MOV AL, WINNER_INDEX ; CARGAR INDICE DEL GANADOR
ADD AL, 30h ; CONVERTIR A CARACTER ASCII
MOV [TEXT_GAME_OVER_WINNER + 7], AL ; GUARDAR EN POSICION DEL TEXTO
RET ; RETORNAR DEL PROCEDIMIENTO
UPDATE_WINNER_TEXT ENDP ; FIN DEL PROCEDIMIENTO

CLEAR_SCREEN PROC NEAR ; PROCEDIMIENTO PARA LIMPIAR LA PANTALLA
MOV AX, 0A000h ; DIRECCION DE MEMORIA DE VIDEO (MOD0
13h)
MOV ES, AX ; GUARDAR EN REGISTRO EXTRA
MOV DI, 0 ; INICIO DEL BUFFER DE VIDEO
MOV AL, 00h ; COLOR NEGRO PARA LIMPIAR
MOV CX, 64000 ; TAMAÑO DEL BUFFER (320*200)
CLD ; LIMPIAR DIRECCION (INCREMENTAR)
REP STOSB ; LLENAR BUFFER CON VALOR DE AL
RET ; RETORNAR DEL PROCEDIMIENTO
CLEAR_SCREEN ENDP ; FIN DEL PROCEDIMIENTO

CODE ENDS
END
```

Listing 1: Main Assembly Code

B. Guía de Sustentación del Proyecto

B.1 El Gancho Inicial (Obligatorio)

Toda presentación debe iniciar con un **“Gancho” (Hook)** de máximo 1 minuto. No inicie diciendo “Hola, somos el grupo X”. Inicie con una pregunta impactante, un dato histórico curioso sobre arquitectura o una demostración visual que capture la atención del público invitado. *Objetivo: Despertar el interés inmediato.*

B.2 Reglas de la Exposición

1. **Tiempo:** 20 a 25 minutos estrictos por grupo.
2. **Diapositivas:** Fondo blanco obligatorio. Menos de 7 líneas de texto por slide. Use muchos diagramas.
3. **Presentación Personal:** Vestimenta formal/profesional acorde a una defensa de ingeniería.
4. **Defensa de la Innovación:** Deben vender su proyecto como la mejor solución. Justifiquen por qué es innovador y por qué es relevante para la clase.

B.3 Rúbrica de Calificación (Informe Final 20 %)

Criterio	Peso	Descripción
Calidad LaTeX	5 %	Uso correcto de la plantilla, ToC, enlaces, figuras y normas IEEE.
Contenido Técnico	5 %	Profundidad en el marco teórico y manejo de conceptos de arquitectura.
Procedimiento	5 %	Replicabilidad: ¿Puedo construir lo mismo leyendo su metodología?
Evidencias	5 %	Capturas de alta calidad y análisis de resultados/discusiones.

B.4 Rúbrica de Calificación (Sustentación 35 %)

Criterio	Peso	Evaluated por
Dominio Técnico	20 %	Profesor: Conocimiento de registros, banderas, flujos de datos y lógica profunda.
Impacto e Innovación	15 %	Público Invitado: Gancho inicial, estética de slides, capacidad de persuasión y relevancia del problema.